

Code Optimization and Data Flow Analysis

Uday Khedker

(www.cse.iitb.ac.in/~uday)

Department of Computer Science and Engineering,
Indian Institute of Technology, Bombay



March 2026



Outline

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Classical optimizations performed by compilers
- Optimizing heap memory usage
- Live variables analysis



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Classical Optimizations



Examples of Optimizing Transformations (ALSU, 2006)

A C program and its optimizations

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

```
void quicksort(int m, int n)
{
    int i, j, v, x;
    if (n <= m) return;
    i = m-1; j = n; v = a[n];           /* v is the pivot */
    while(1)                             /* Move values smaller
    {                                     /* than v to the left of
        do i = i + 1; while (a[i] < v); /* the split point (sp)
        do j = j - 1; while (a[j] > v); /* and other values
        if (i >= j) break;               /* to the right of sp
        x = a[i]; a[i] = a[j]; a[j] = x; /* of the split point
    }                                     /* Move the pivot to sp
    x = a[i]; a[i] = a[n]; a[n] = x;     /* sort the partitions to
    quicksort(m,i); quicksort(i+1,n);    /* the left of sp and to the right of sp independently
}
```



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Intermediate Code

For the boxed source code

1. $i = m - 1$

2. $j = n$

3. $t1 = 4 * n$

4. $t6 = a[t1]$

5. $v = t6$

6. $i = i + 1$

7. $t2 = 4 * i$

8. $t3 = a[t2]$

9. if $t3 < v$ goto 6

10. $j = j - 1$

11. $t4 = 4 * j$

12. $t5 = a[t4]$

13. if $t5 > v$ goto 10

14. if $i \geq j$ goto 25

15. $t2 = 4 * i$

16. $t3 = a[t2]$

17. $x = t3$

18. $t2 = 4 * i$

19. $t4 = 4 * j$

20. $t5 = a[t4]$

21. $a[t2] = t5$

22. $t4 = 4 * j$

23. $a[t4] = x$

24. goto 6

25. $t2 = 4 * i$

26. $t3 = a[t2]$

27. $x = t3$

28. $t2 = 4 * i$

29. $t1 = 4 * n$

30. $t6 = a[t1]$

31. $a[t2] = t6$

32. $t1 = 4 * n$

33. $a[t1] = x$



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Intermediate Code : Observations

```
1. i = m - 1
2. j = n
3. t1 = 4 * n
4. t6 = a[t1]
5. v = t6
6. i = i + 1
7. t2 = 4 * i
8. t3 = a[t2]
9. if t3 < v goto 6
10. j = j - 1
11. t4 = 4 * j
12. t5 = a[t4]
13. if t5 > v goto 10
14. if i >= j goto 25
15. t2 = 4 * i
16. t3 = a[t2]
17. x = t3
18. t2 = 4 * i
19. t4 = 4 * j
20. t5 = a[t4]
21. a[t2] = t5
22. t4 = 4 * j
23. a[t4] = x
24. goto 6
25. t2 = 4 * i
26. t3 = a[t2]
27. x = t3
28. t2 = 4 * i
29. t1 = 4 * n
30. t6 = a[t1]
31. a[t2] = t6
32. t1 = 4 * n
33. a[t1] = x
```

- Three address code (at most three variables involved in any statement)
Temporary variables used to break down expressions into smaller expressions
- Multiple computations of an expression stored in the same temporary
(only multiplication and array access expressions shown in the picture)
- Simple control flow
(conditional/unconditional goto)
Yet undecipherable!



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Understanding Control Flow

- Identify maximal sequences of linear control flow
⇒ Basic Blocks
- No transfer into or out of basic blocks except the first and last statements
Control transfer into the block : only at the first statement.
Control transfer out of the block : only at the last statement.



Intermediate Code with Basic Blocks

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

1. $i = m - 1$

2. $j = n$

3. $t1 = 4 * n$

4. $t6 = a[t1]$

5. $v = t6$

6. $i = i + 1$

7. $t2 = 4 * i$

8. $t3 = a[t2]$

9. if $t3 < v$ goto 6

10. $j = j - 1$

11. $t4 = 4 * j$

12. $t5 = a[t4]$

13. if $t5 > v$ goto 10

14. if $i \geq j$ goto 25

15. $t2 = 4 * i$

16. $t3 = a[t2]$

17. $x = t3$

18. $t2 = 4 * i$

19. $t4 = 4 * j$

20. $t5 = a[t4]$

21. $a[t2] = t5$

22. $t4 = 4 * j$

23. $a[t4] = x$

24. goto 6

25. $t2 = 4 * i$

26. $t3 = a[t2]$

27. $x = t3$

28. $t2 = 4 * i$

29. $t1 = 4 * n$

30. $t6 = a[t1]$

31. $a[t2] = t6$

32. $t1 = 4 * n$

33. $a[t1] = x$



IIT Bombay
cs302: Implementation
of Programming
Languages

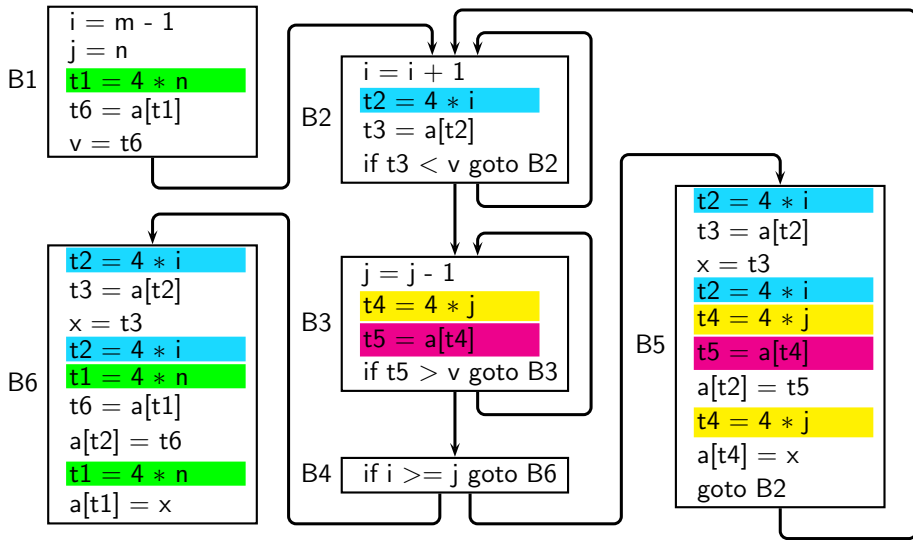
Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Program Flow Graph





Program Flow Graph : Observations

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Nesting Level	Basic Blocks	No. of Statements
0	B1, B6	14
1	B4, B5	11
2	B2, B3	8



IIT Bombay
cs302: Implementation
of Programming
Languages

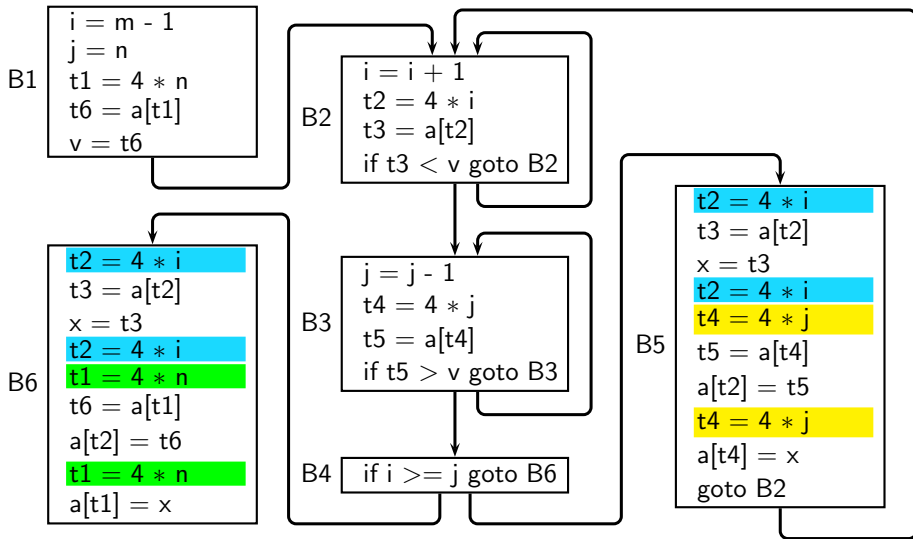
Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Local Common Subexpression Elimination

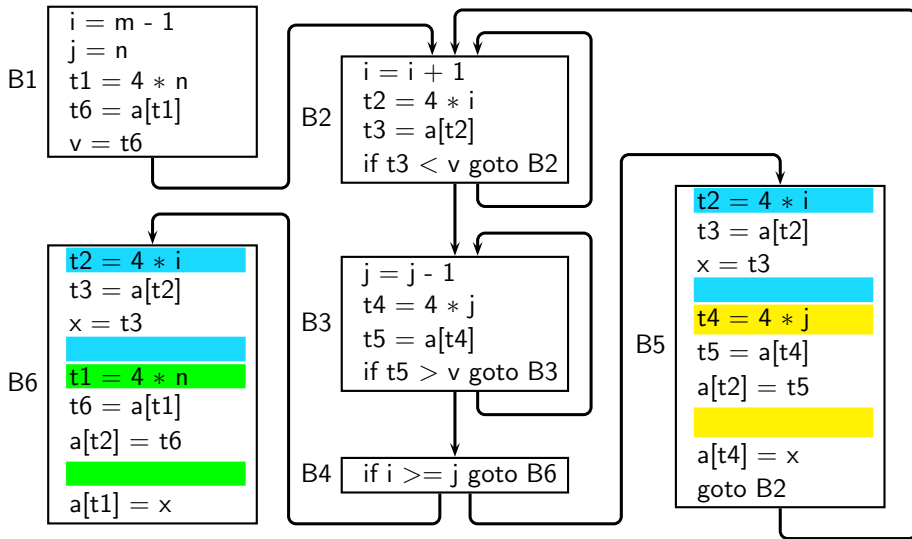




IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis
Section:
Classical
Optimizations
Optimizing Heap
Memory Usage
Live Variables
Analysis

Local Common Subexpression Elimination





IIT Bombay
cs302: Implementation
of Programming
Languages

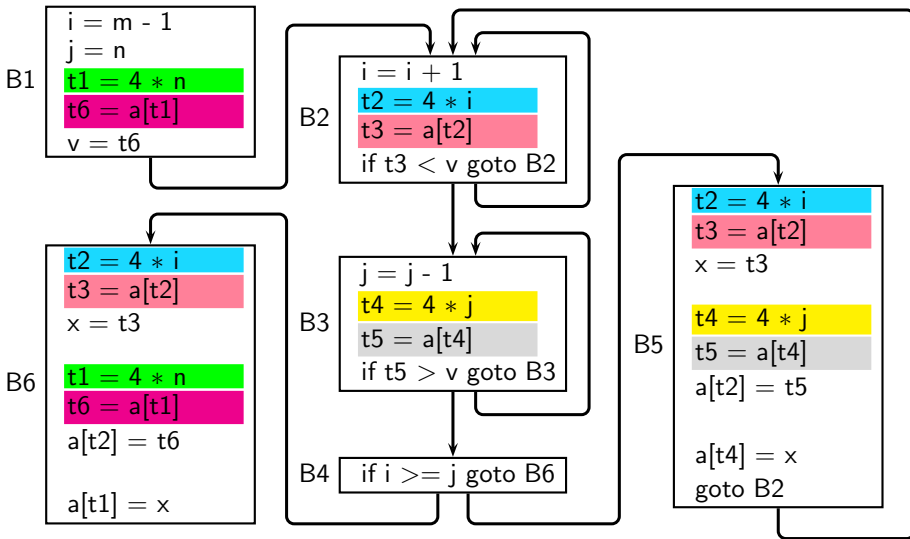
Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Global Common Subexpression Elimination





Global Common Subexpression Elimination

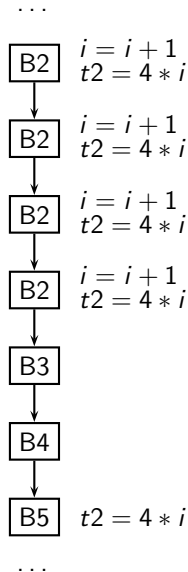
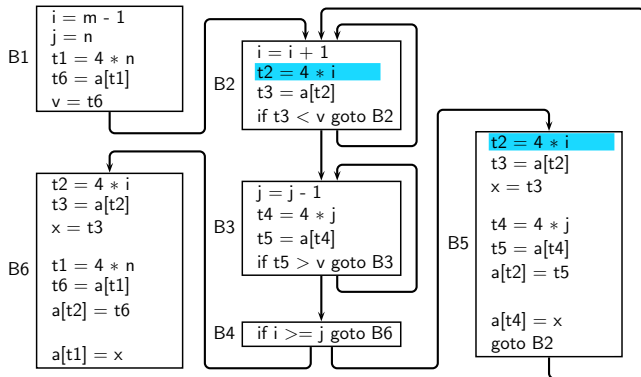
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Global Common Subexpression Elimination

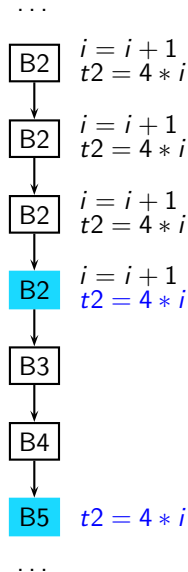
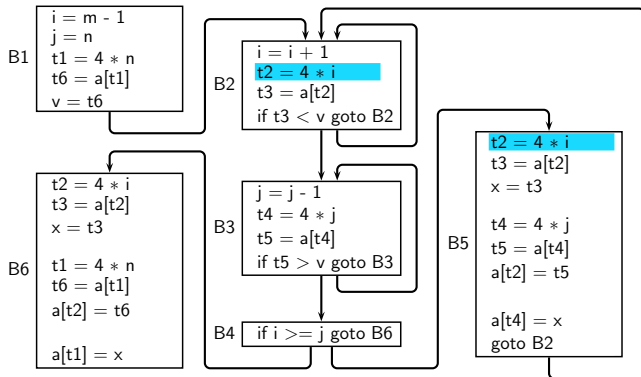
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





IIT Bombay
cs302: Implementation
of Programming
Languages

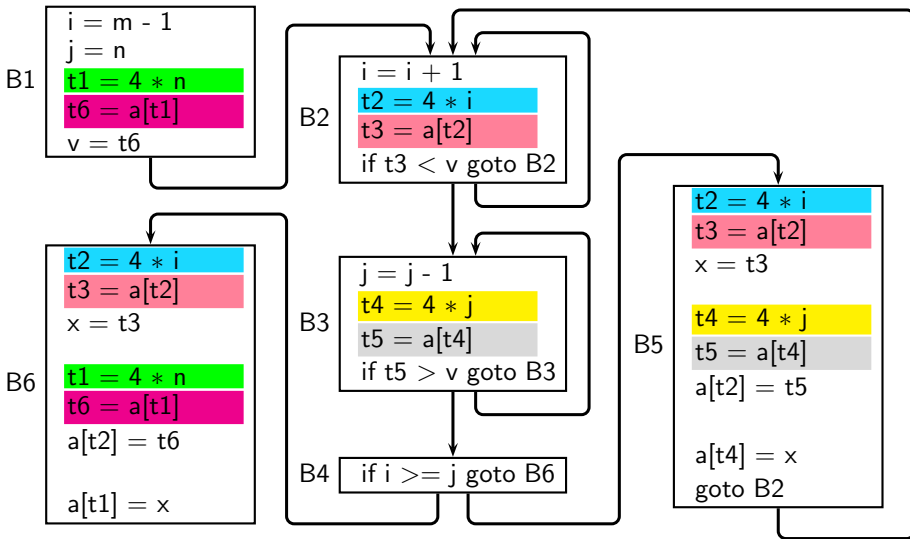
Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Global Common Subexpression Elimination





IIT Bombay
cs302: Implementation
of Programming
Languages

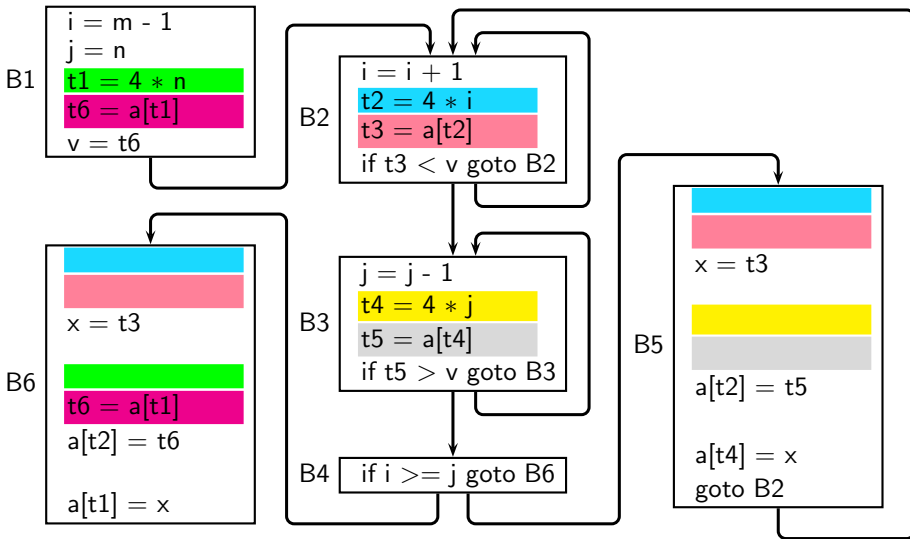
Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Global Common Subexpression Elimination





Other Classical Optimizations

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Copy propagation
- Strength Reduction
- Elimination of Induction Variables
- Dead Code Elimination

Copy Propagation and Dead Code Elimination



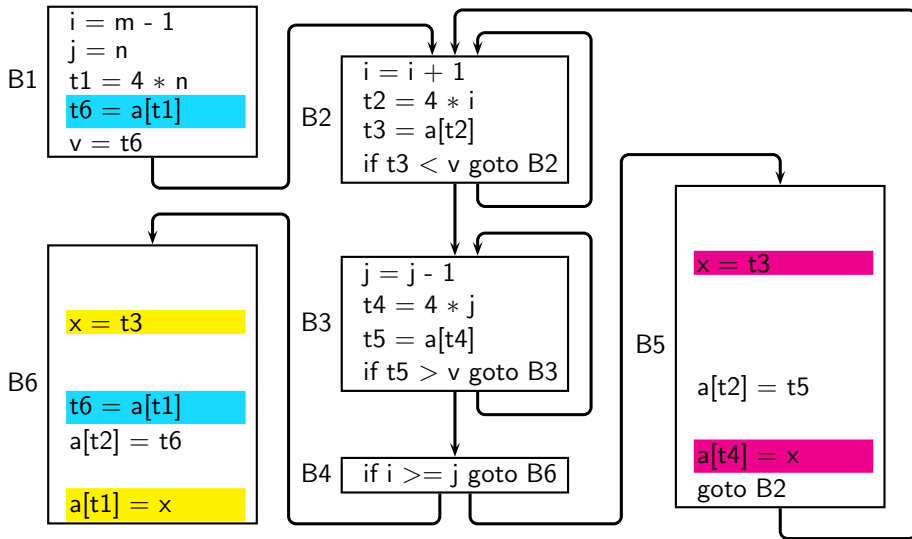
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



Copy Propagation and Dead Code Elimination



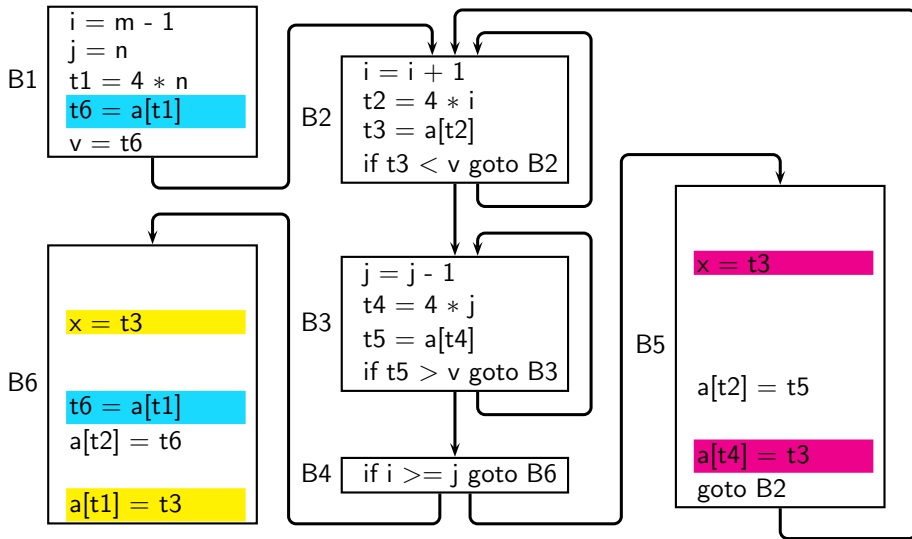
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



Copy Propagation and Dead Code Elimination



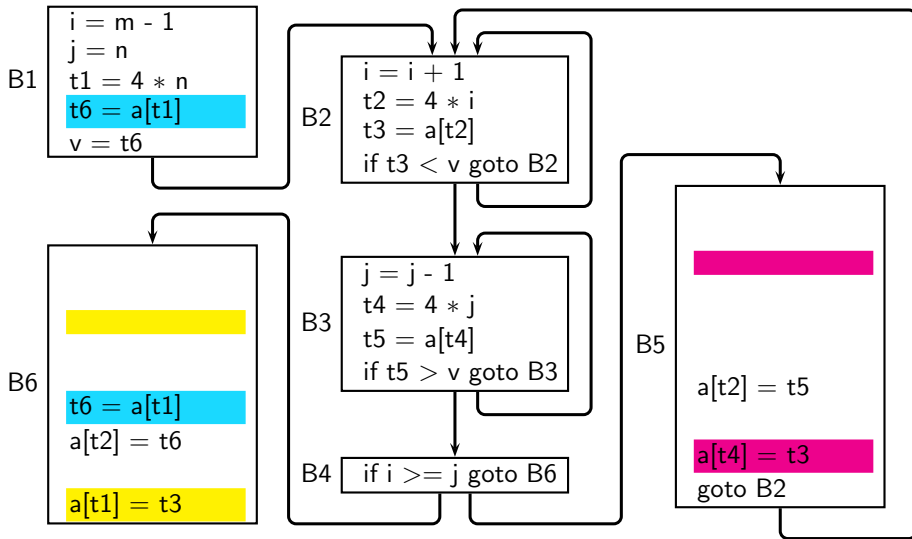
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Strength Reduction and Induction Variable Elimination

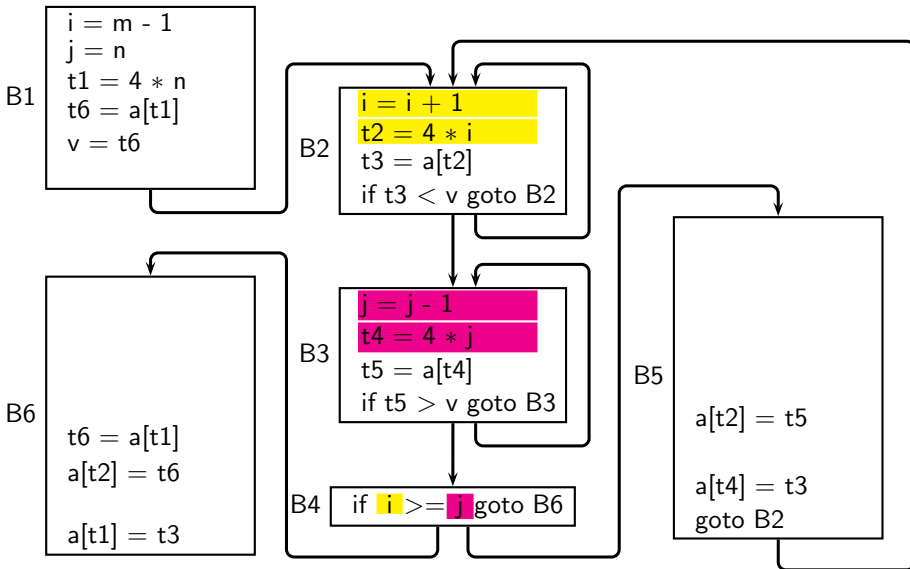
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Strength Reduction and Induction Variable Elimination

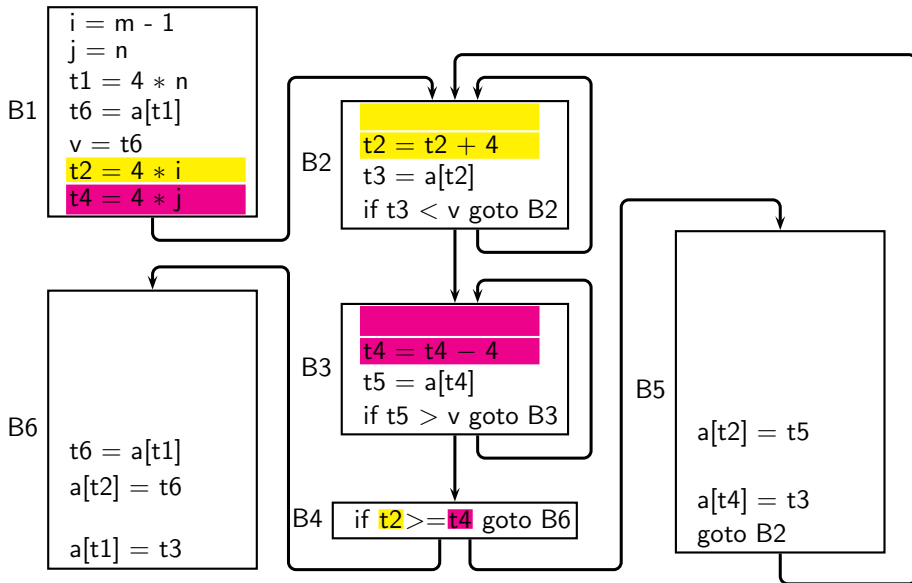
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

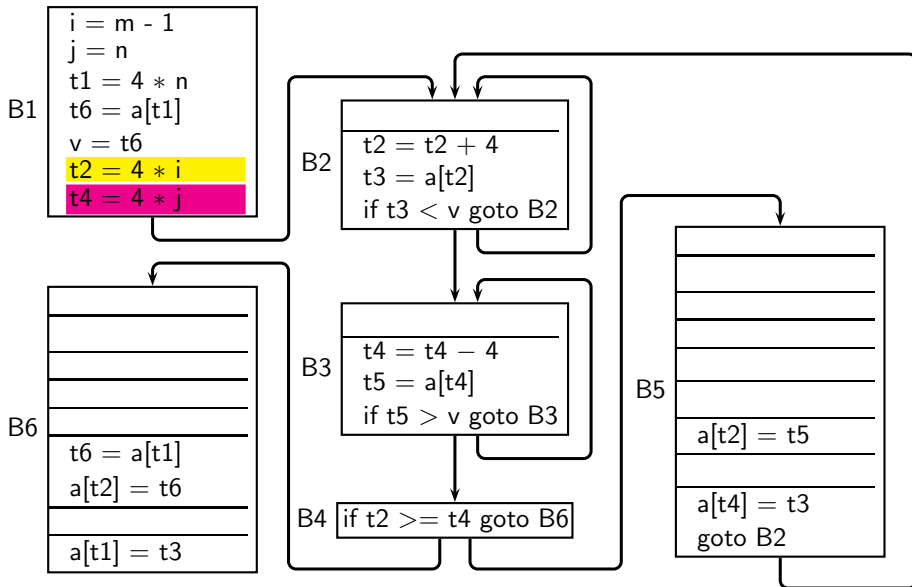
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Final Intermediate Code





Optimized Program Flow Graph

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Nesting Level	No. of Statements	
	Original	Optimized
0	14	10
1	11	4
2	8	6

If we assume that a loop is executed 10 times, then the number of computations saved at run time

$$= (14 - 10) + (11 - 4) \times 10 + (8 - 6) \times 10^2 = 4 + 70 + 200 = 274$$



Observations

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Optimizations are transformations based on some information.
- Systematic analysis required for deriving the information.
- We have looked at data flow optimizations.
Many control flow optimizations can also be performed.



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Categories of Optimizing Transformations and Analyses

Code Motion Redundancy Elimination Control flow Optimization	Machine Independent	Flow Analysis (Data + Control)
Loop Transformations	Machine Dependent	Dependence Analysis (Data + Control)
Instruction Scheduling Register Allocation Peephole Optimization	Machine Dependent	Several Independent Techniques
Vectorization Parallelization	Machine Dependent	Dependence Analysis (Data + Control)



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Optimizing Heap Memory Usage



Standard Memory Architecture of Programs

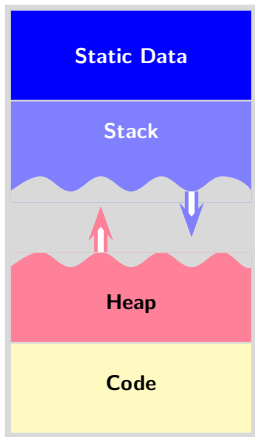
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



Heap allocation provides the flexibility of

- *Variable Sizes*. Data structures can grow or shrink as desired at runtime.
(Not bound to the declarations in program.)
- *Variable Lifetimes*. Data structures can be created and destroyed as desired at runtime.
(Not bound to the activations of procedures.)



Managing Heap Memory

Decision 1: When to Allocate?

- **Explicit.** Specified in the programs. (eg. Imperative/OO languages)
- **Implicit.** Decided by the language processors. (eg. Declarative Languages)

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



Managing Heap Memory

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Decision 1: When to Allocate?

- **Explicit.** Specified in the programs. (eg. Imperative/OO languages)
- **Implicit.** Decided by the language processors. (eg. Declarative Languages)

Decision 2: When to Deallocate?

- **Explicit.** Manual Memory Management (eg. C/C++)
- **Implicit.** Automatic Memory Management aka Garbage Collection (eg. Java/Declarative languages)



State of Art in Manual Deallocation

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Memory leaks
10% to 20% of last development effort goes in plugging leaks
- Tool assisted manual plugging
Purify, Electric Fence, RootCause, GlowCode, yakTest, Leak Tracer, BDW Garbage Collector, mtrace, memwatch, dmalloc etc.
- All leak detectors
 - are dynamic (and hence specific to execution instances)
 - generate massive reports to be perused by programmers
 - usually do not locate last use but only allocation escaping a call
⇒ At which program point should a leak be “plugged”?

Garbage Collection $\equiv$ Automatic Deallocation



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Retain active data structure.
Deallocation inactive data structure.
- What is an Active Data Structure?

Garbage Collection $\equiv$ Automatic Deallocation



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Retain active data structure.
Deallocate inactive data structure.
- What is an Active Data Structure?

If an object does not have an access path, (i.e. it is unreachable)
then its memory can be reclaimed.

Garbage Collection $\equiv$ Automatic Deallocation



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Retain active data structure.
Deallocate inactive data structure.
- What is an Active Data Structure?

If an object does not have an access path, (i.e. it is unreachable)
then its memory can be reclaimed.

What if an object has an access path, but is not accessed after the given program point?

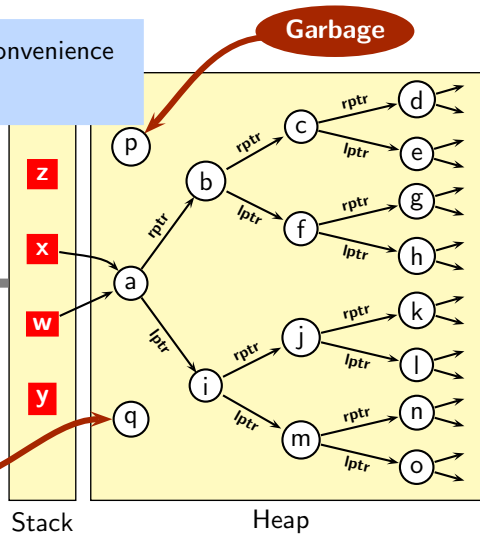


What is Garbage?

We use Java style statements for convenience

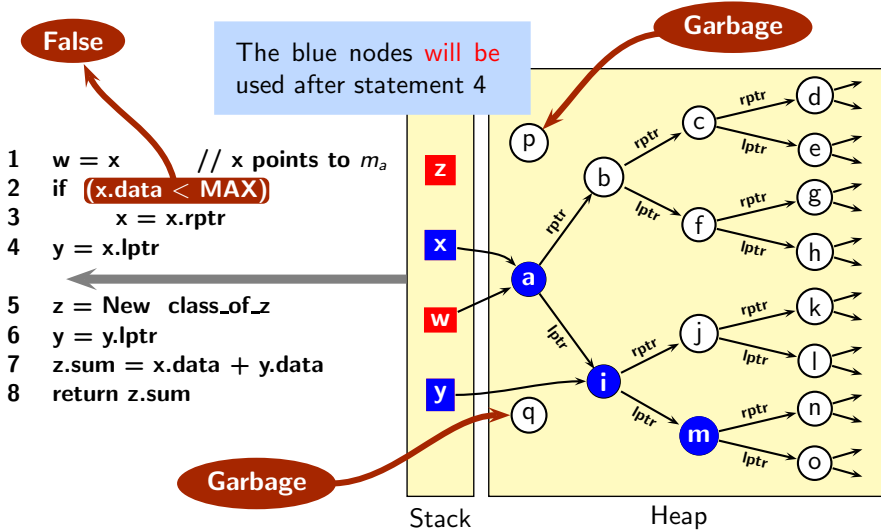
Read “x.lptr” as “x→lptr

```
1  w = x      // x points to ma
2  if (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```



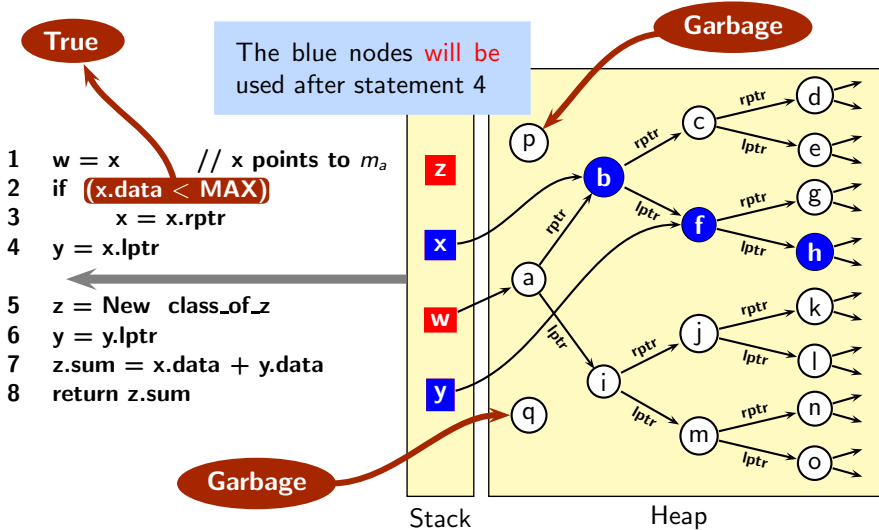


What is Garbage?





What is Garbage?

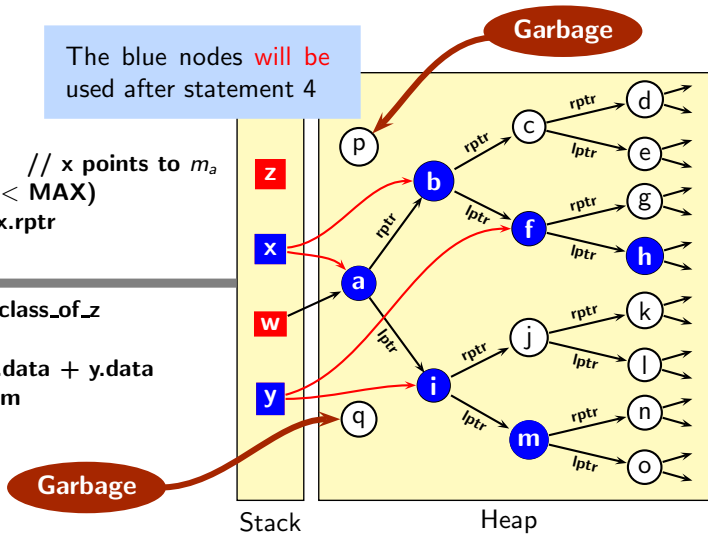




What is Garbage?

```
1  w = x      // x points to ma
2  if (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```

The blue nodes will be used after statement 4



All white nodes are unused and should be considered garbage



Is Reachable Same as Live?

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

From www.memorymanagement.org/glossary

live (also known as alive, active) : Memory(2) or an object is live if the program will read from it in future. *The term is often used more broadly to mean reachable.*

It is not possible, in general, for garbage collectors to determine exactly which objects are still live. Instead, they use some approximation to detect objects that are provably dead, *such as those that are not reachable.*

Similar terms: reachable. Opposites: dead. See also: undead.



Is Reachable Same as Live?

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Not really. Most of us know that.

Even with the state of art of garbage collection, 24% to 76% unused memory remains unclaimed

- The state of art compilers, virtual machines, garbage collectors cannot distinguish between the two



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

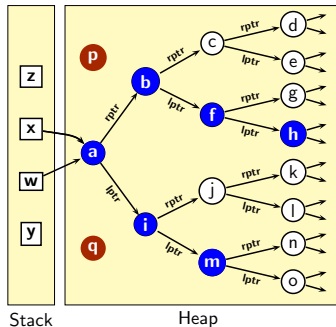
Live Variables
Analysis

Reachability and Liveness

Some unused memory remains unclaimed because garbage collectors collect unreachable memory and not unused (i.e. non-live) memory

For the heap memory on the right

Allocated	White + Blue + Brown nodes
Reachable	White + Blue nodes
Live	Blue nodes





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

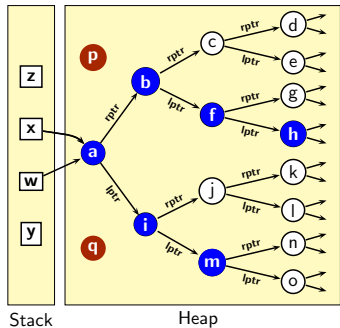
Live Variables
Analysis

Reachability and Liveness

Some unused memory remains unclaimed because garbage collectors collect unreachable memory and not unused (i.e. non-live) memory

For the heap memory on the right

Allocated	White + Blue + Brown nodes
Reachable	White + Blue nodes
Live	Blue nodes



Live $\subseteq$ Reachable $\subseteq$ Allocated

Hence, $\neg\text{Live} \supseteq \neg\text{Reachable} \supseteq \neg\text{Allocated}$



Cedar Mesa Folk Wisdom

Make the unused memory unreachable by setting references to NULL. (GC FAQ:
<http://www.iecc.com/gclist/GC-harder.html>)

IIT Bombay
cs302: Implementation
of Programming
Languages

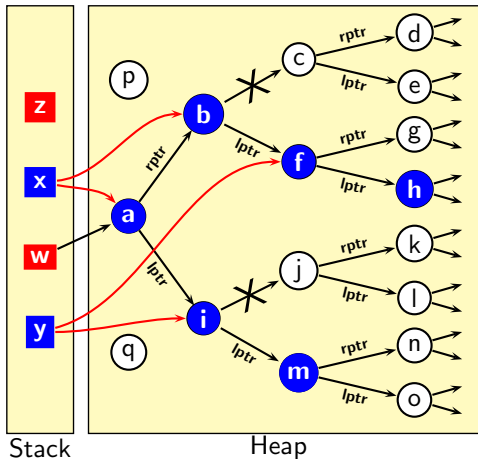
Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Cedar Mesa Folk Wisdom

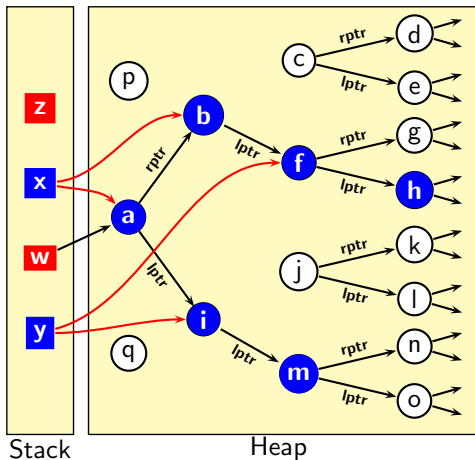
Make the unused memory unreachable by setting references to NULL. (GC FAQ:
<http://www.iecc.com/gclist/GC-harder.html>)

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations
Optimizing Heap
Memory Usage

Live Variables
Analysis





Cedar Mesa Folk Wisdom

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Most promising, simplest to understand, yet the hardest to implement.
- Which references should be set to NULL?
 - Most approaches rely on feedback from profiling.
 - No systematic and clean solution.



Distinguishing Between Reachable and Live

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

The state of art

- Eliminating objects reachable from root variables which are not live.
- Uses liveness data flow analysis of root variables (stack data).
- What about liveness of heap data?

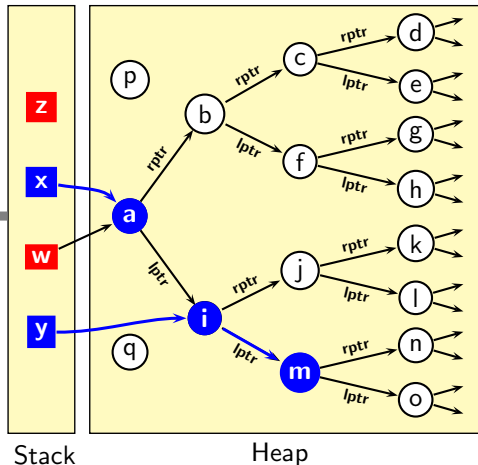


Applying Cedar Mesa Folk Wisdom to Heap Data

Liveness Analysis of Heap Data

If the **while** loop is not executed even once.

```
1  w = x      // x points to ma
2  while (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```



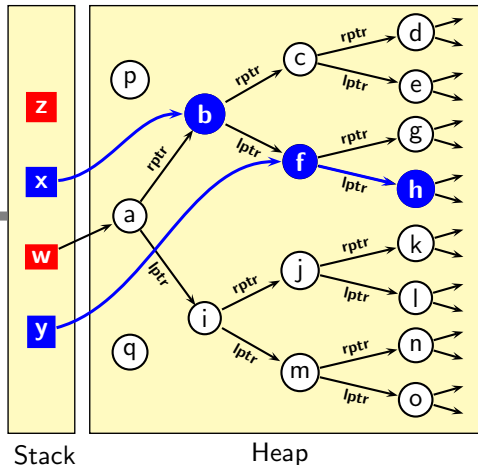


Applying Cedar Mesa Folk Wisdom to Heap Data

Liveness Analysis of Heap Data

If the **while** loop is executed once.

```
1  w = x      // x points to ma
2  while (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```



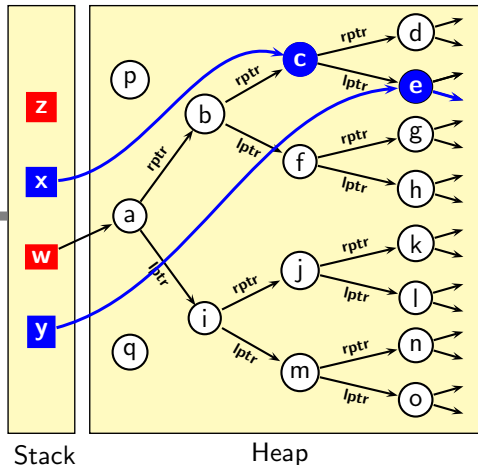


Applying Cedar Mesa Folk Wisdom to Heap Data

Liveness Analysis of Heap Data

If the **while** loop is executed twice.

```
1  w = x      // x points to ma
2  while (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

The Moral of the Story

- Mappings between access expressions and l-values keep changing
- This is a *rule* for heap data
For stack and static data, it is an *exception*!
- Static analysis of programs has made significant progress for stack and static data.

What about heap data?

- Given two access expressions at a program point, do they have the same l-value?
- Given the same access expression at two program points, does it have the same l-value?



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Our Solution (1)

```

                                y = z = null
1  w = x
                                w = null
2  while (x.data < MAX)
    {
3      x = x.rptr      }
                                x.rptr = x.lptr.rptr = null
                                x.lptr.lptr.lptr = x.lptr.lptr.rptr = null
4  y = x.lptr
                                x.lptr = y.rptr = null
                                y.lptr.lptr = y.lptr.rptr = null
5  z = New class_of_z
                                z.lptr = z.rptr = null
6  y = y.lptr
                                y.lptr = y.rptr = null
7  z.sum = x.data + y.data
                                x = y = null
8  return z.sum
                                z = null
```



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

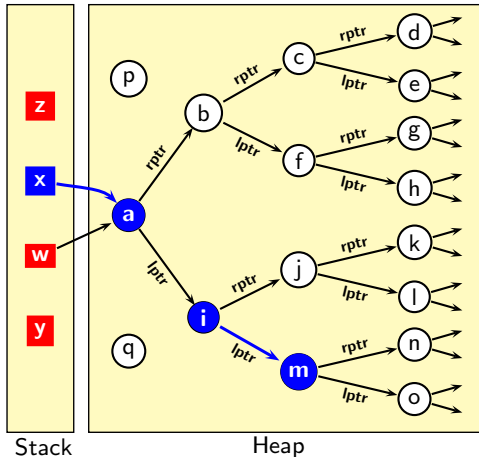
Optimizing Heap
Memory Usage

Live Variables
Analysis

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  {   x.lptr = null
3     x = x.rptr   }
    x.rptr = x.lptr.rptr = null
    x.lptr.lptr.lptr = null
    x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

Our Solution (2)

While loop is not executed even once





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

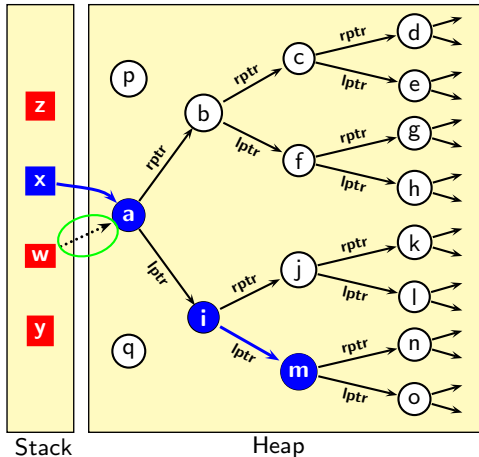
Optimizing Heap
Memory Usage

Live Variables
Analysis

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  {   x.lptr = null
3     x = x.rptr   }
  x.rptr = x.lptr.rptr = null
  x.lptr.lptr.lptr = null
  x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

Our Solution (2)

While loop is not executed even once





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

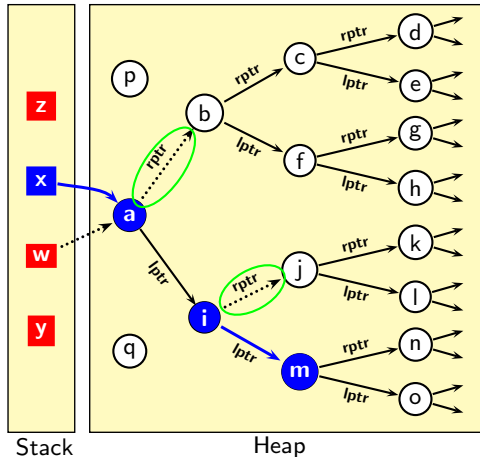
```

y = z = null
1  w = x
   w = null
2  while (x.data < MAX)
   {   x.lptr = null
3      x = x.rptr   }
   x.rptr = x.lptr.rptr = null
   x.lptr.lptr.lptr = null
   x.lptr.lptr.rptr = null
4  y = x.lptr
   x.lptr = y.rptr = null
   y.lptr.lptr = y.lptr.rptr = null
5  z = New class_of_z
   z.lptr = z.rptr = null
6  y = y.lptr
   y.lptr = y.rptr = null
7  z.sum = x.data + y.data
   x = y = null
8  return z.sum
   z = null

```

Our Solution (2)

While loop is not executed even once





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

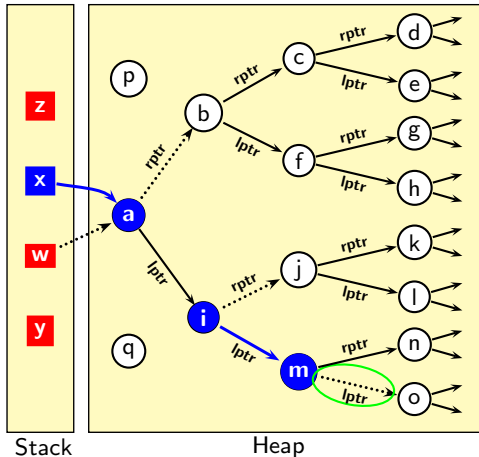
Optimizing Heap
Memory Usage

Live Variables
Analysis

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  {   x.lptr = null
3     x = x.rptr   }
    x.rptr = x.lptr.rptr = null
    x.lptr.lptr.lptr = null
    x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

Our Solution (2)

While loop is not executed even once





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

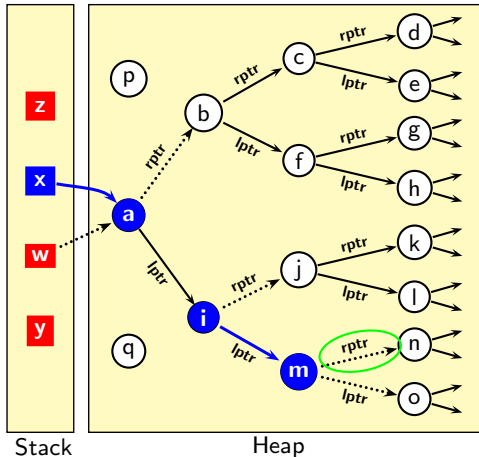
Optimizing Heap
Memory Usage

Live Variables
Analysis

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  {   x.lptr = null
3     x = x.rptr   }
    x.rptr = x.lptr.rptr = null
    x.lptr.lptr.lptr = null
    x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

Our Solution (2)

While loop is not executed even once





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

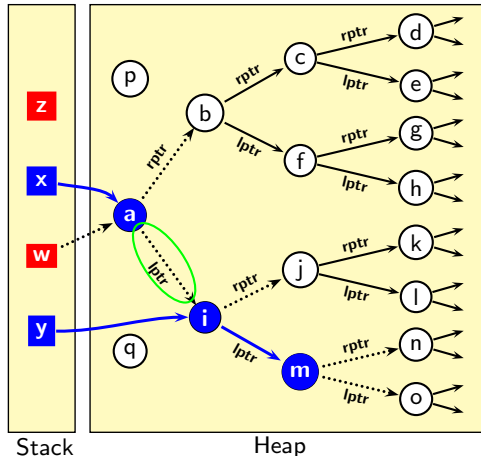
```

y = z = null
1  w = x
   w = null
2  while (x.data < MAX)
   {   x.lptr = null
3      x = x.rptr   }
   x.rptr = x.lptr.rptr = null
   x.lptr.lptr.lptr = null
   x.lptr.lptr.rptr = null
4  y = x.lptr
   x.lptr = y.rptr = null
   y.lptr.lptr = y.lptr.rptr = null
5  z = New class_of_z
   z.lptr = z.rptr = null
6  y = y.lptr
   y.lptr = y.rptr = null
7  z.sum = x.data + y.data
   x = y = null
8  return z.sum
   z = null

```

Our Solution (2)

While loop is not executed even once





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

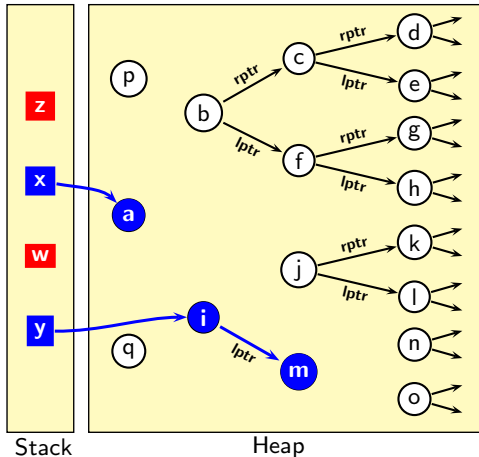
Optimizing Heap
Memory Usage

Live Variables
Analysis

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  {   x.lptr = null
3     x = x.rptr   }
    x.rptr = x.lptr.rptr = null
    x.lptr.lptr.lptr = null
    x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

Our Solution (2)

While loop is not executed even once





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

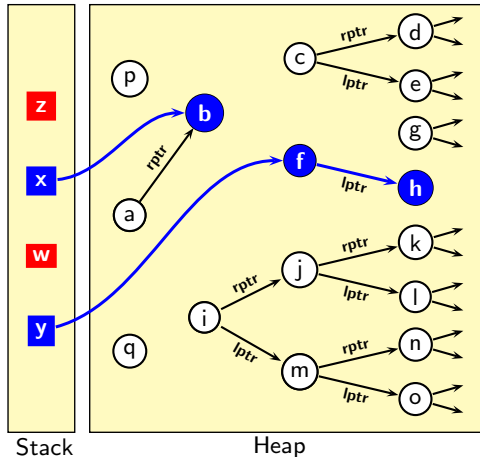
Optimizing Heap
Memory Usage

Live Variables
Analysis

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  {   x.lptr = null
3     x = x.rptr   }
  x.rptr = x.lptr.rptr = null
  x.lptr.lptr.lptr = null
  x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

Our Solution (2)

While loop is executed once





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

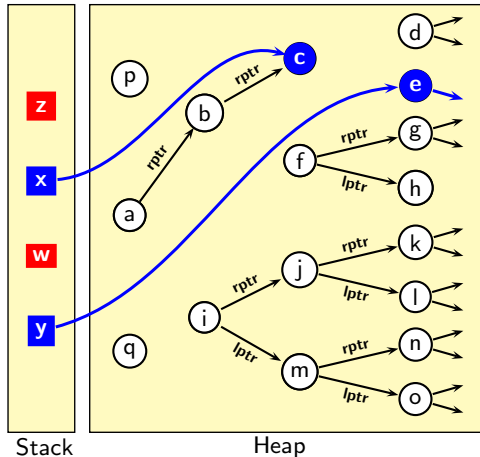
Optimizing Heap
Memory Usage

Live Variables
Analysis

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  {   x.lptr = null
3     x = x.rptr      }
    x.rptr = x.lptr.rptr = null
    x.lptr.lptr.lptr = null
    x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

Our Solution (2)

While loop is executed twice





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

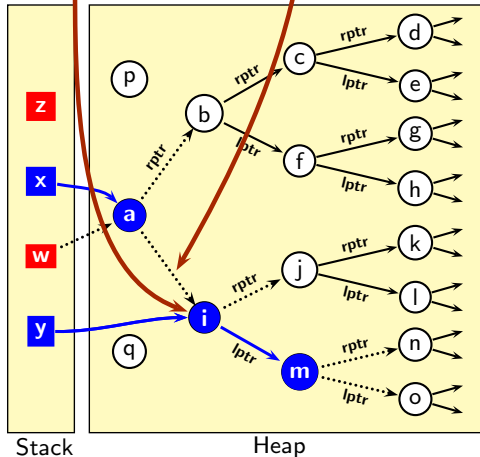
Optimizing Heap
Memory Usage

Live Variables
Analysis

Some Observations

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  { x.lptr = null
3    x = x.rptr    }
  x.rptr = x.lptr.rptr = null
  x.lptr.lptr.lptr = null
  x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

Node i is live but link $a \rightarrow i$ is nullified





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

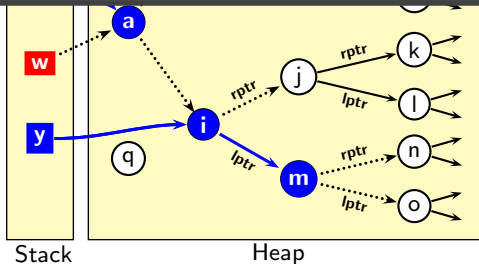
Optimizing Heap
Memory Usage

Live Variables
Analysis

Some Observations

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  {   x.lptr = null
3     x = x.rptr   }
    x.rptr = x.lptr.rptr = null
    x.lptr.lptr.lptr = null
    x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

- The memory address that x holds when the execution reaches a given program point is not an invariant of program execution





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

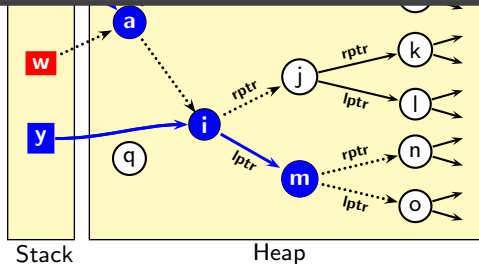
Optimizing Heap
Memory Usage

Live Variables
Analysis

Some Observations

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  {   x.lptr = null
3     x = x.rptr   }
    x.rptr = x.lptr.rptr = null
    x.lptr.lptr.lptr = null
    x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

- The memory address that x holds when the execution reaches a given program point is not an invariant of program execution
- Whether we dereference $lptr$ out of x or $rptr$ out of x at a given program point is an invariant of program execution





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

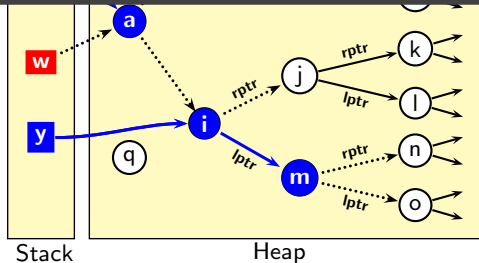
Optimizing Heap
Memory Usage

Live Variables
Analysis

Some Observations

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  {   x.lptr = null
3     x = x.rptr   }
    x.rptr = x.lptr.rptr = null
    x.lptr.lptr.lptr = null
    x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

- The memory address that x holds when the execution reaches a given program point is not an invariant of program execution
- Whether we dereference $lptr$ out of x or $rptr$ out of x at a given program point is an invariant of program execution
- *A static analysis can discover only invariants*





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

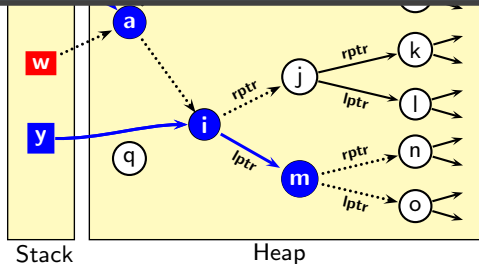
Optimizing Heap
Memory Usage

Live Variables
Analysis

Some Observations

```
y = z = null
1 w = x
  w = null
2 while (x.data < MAX)
  {   x.lptr = null
3     x = x.rptr   }
    x.rptr = x.lptr.rptr = null
    x.lptr.lptr.lptr = null
    x.lptr.lptr.rptr = null
4 y = x.lptr
  x.lptr = y.rptr = null
  y.lptr.lptr = y.lptr.rptr = null
5 z = New class_of_z
  z.lptr = z.rptr = null
6 y = y.lptr
  y.lptr = y.rptr = null
7 z.sum = x.data + y.data
  x = y = null
8 return z.sum
  z = null
```

- The memory address that x holds when the execution reaches a given program point is not an invariant of program execution
- Whether we dereference $lptr$ out of x or $rptr$ out of x at a given program point is an invariant of program execution
- *A static analysis can discover only some invariants*





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Live Variables Analysis



Liveness of Stack Data: An Informal Introduction (1)

We use Java style statements for convenience

Read “**x.lptr**” as “**x**→**lptr**”

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

```
1  w = x          // x points to  $m_a$ 
2  while (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```



Heap



Stack

if changed to while



Liveness of Stack Data: An Informal Introduction (1)

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

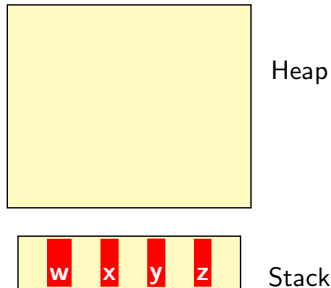
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

```
1  w = x          // x points to  $m_a$ 
2  while (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```



*What is the
meaning of the **use**
of data?*



Liveness of Stack Data: An Informal Introduction (1)

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

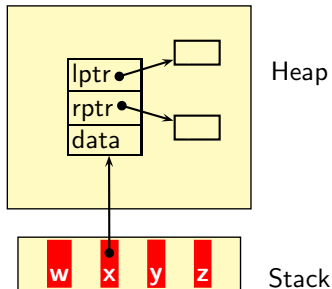
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

```
1  w = x          // x points to  $m_a$ 
2  while (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```



*What is the
meaning of the **use**
of data?*



Liveness of Stack Data: An Informal Introduction (1)

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

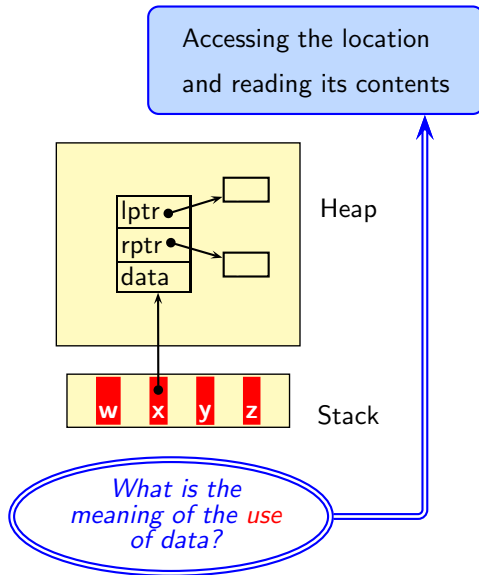
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

```
1  w = x      // x points to  $m_a$ 
2  while (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```





Liveness of Stack Data: An Informal Introduction (1)

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

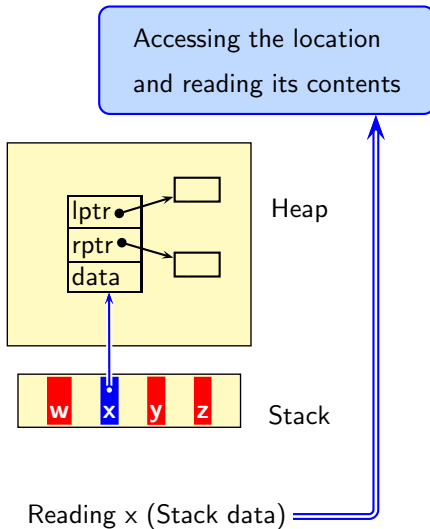
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

```
1  w = x      // x points to ma
2  while (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```





Liveness of Stack Data: An Informal Introduction (1)

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

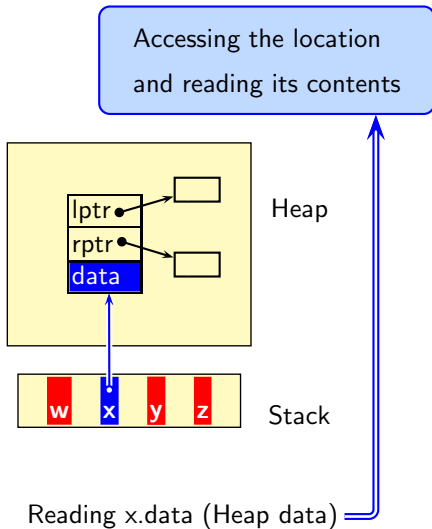
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

```
1  w = x          // x points to  $m_a$ 
2  while (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```





Liveness of Stack Data: An Informal Introduction (1)

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

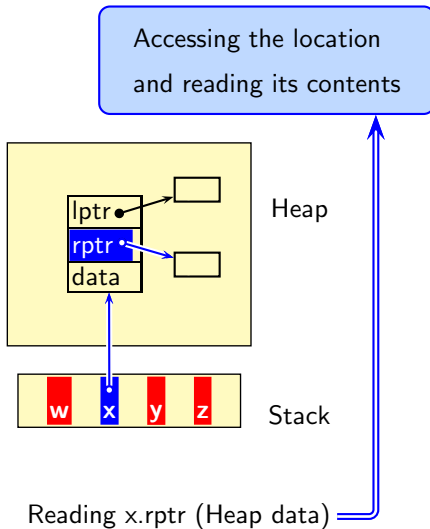
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

```
1  w = x          // x points to  $m_a$ 
2  while (x.data < MAX)
3      x = x.rptr
4  y = x.lptr
5  z = New class_of_z
6  y = y.lptr
7  z.sum = x.data + y.data
8  return z.sum
```





Liveness of Stack Data: An Informal Introduction (2)

IIT Bombay
cs302: Implementation
of Programming
Languages

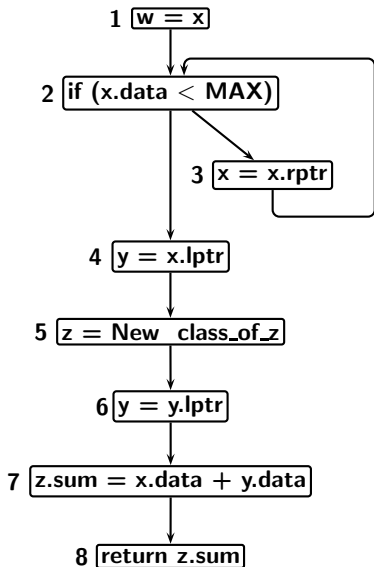
Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



No variable is used beyond this
program point





Liveness of Stack Data: An Informal Introduction (2)

IIT Bombay
cs302: Implementation
of Programming
Languages

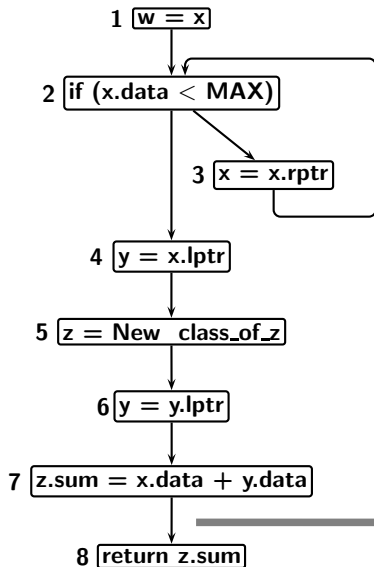
Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



Current value of z is used beyond
this program point



Live

Dead



Liveness of Stack Data: An Informal Introduction (2)

IIT Bombay
cs302: Implementation
of Programming
Languages

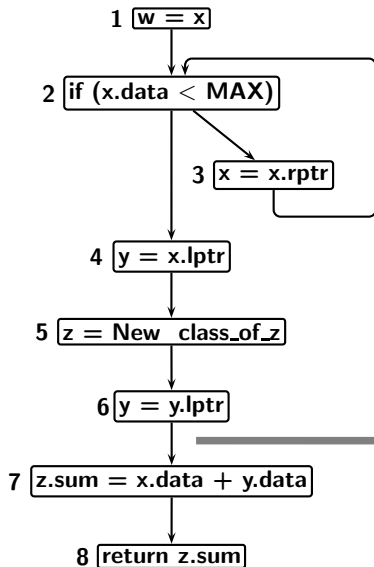
Topic:
Data Flow Analysis

Section:

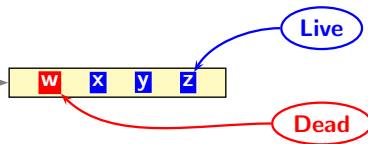
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



Current values of x, y, and z are
used beyond this program point





Liveness of Stack Data: An Informal Introduction (2)

IIT Bombay
cs302: Implementation
of Programming
Languages

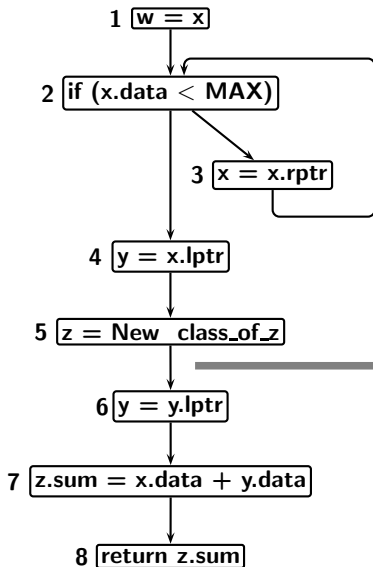
Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

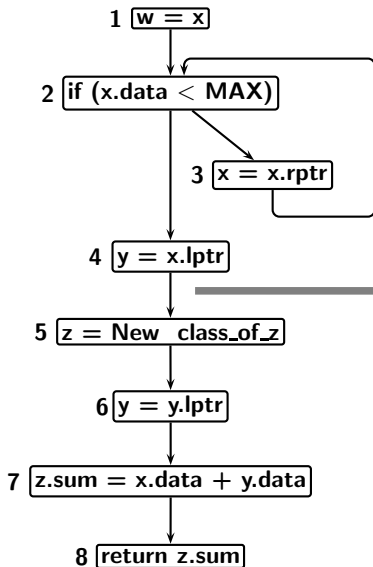


- Current values of x, y, and z are used beyond this program point
- The value of y is different before and after the assignment to y

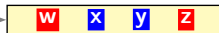




Liveness of Stack Data: An Informal Introduction (2)



- The current values of x and y are used beyond this program point
- The current value of z is not used beyond this program point



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



Liveness of Stack Data: An Informal Introduction (2)

IIT Bombay
cs302: Implementation
of Programming
Languages

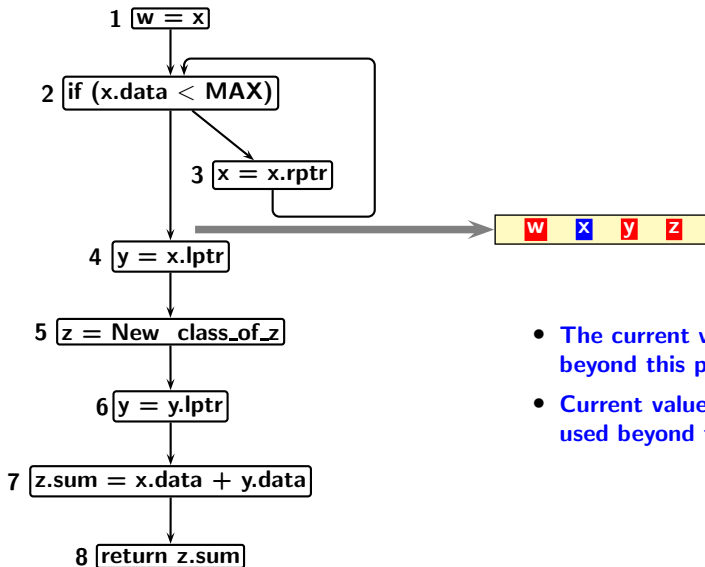
Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



- The current values of x is used beyond this program point
- Current values of y and z are not used beyond this program point



Liveness of Stack Data: An Informal Introduction (2)

IIT Bombay
cs302: Implementation
of Programming
Languages

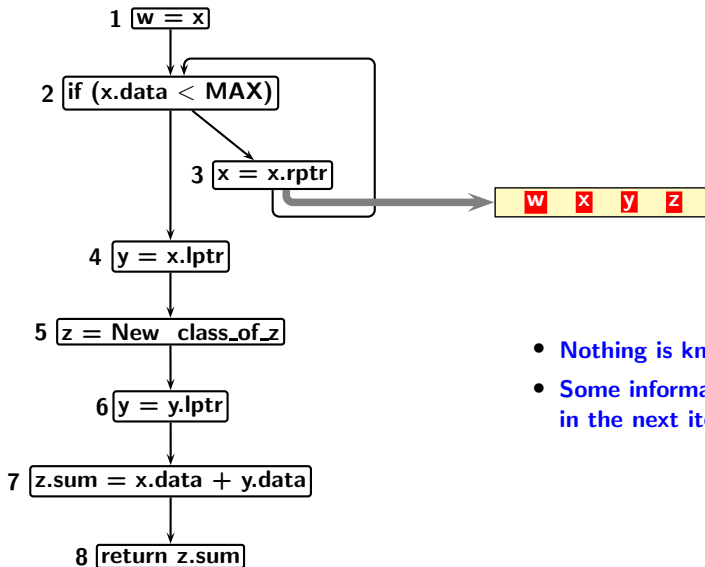
Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



- Nothing is known as of now
- Some information will be available in the next iteration point



Liveness of Stack Data: An Informal Introduction (2)

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

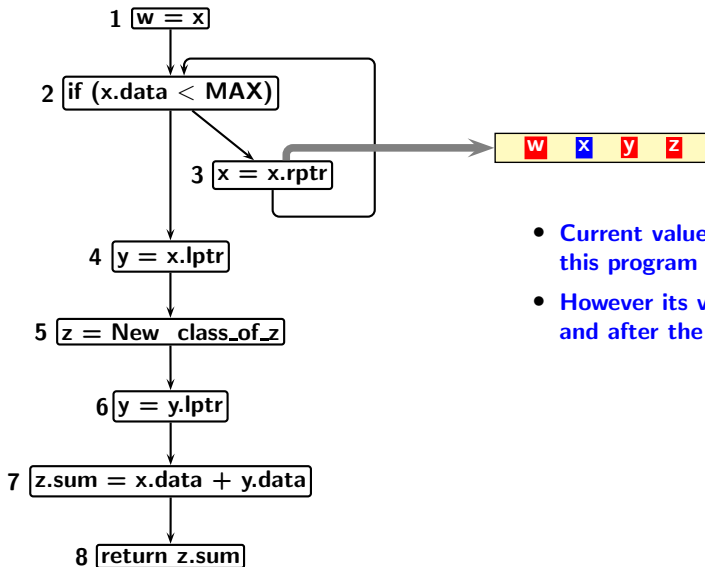
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



- Current value of x is used beyond this program point
- However its value is different before and after the assignment



Liveness of Stack Data: An Informal Introduction (2)

IIT Bombay
cs302: Implementation
of Programming
Languages

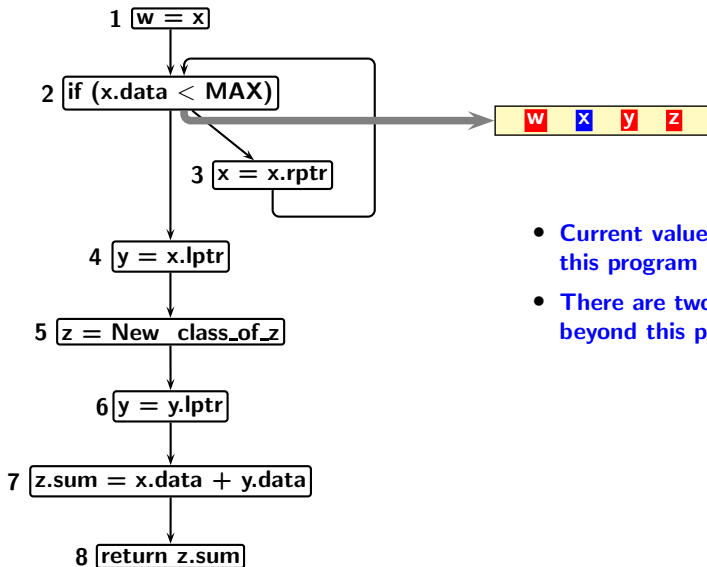
Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



- Current value of x is used beyond this program point
- There are two control flow paths beyond this program point



Liveness of Stack Data: An Informal Introduction (2)

IIT Bombay
cs302: Implementation
of Programming
Languages

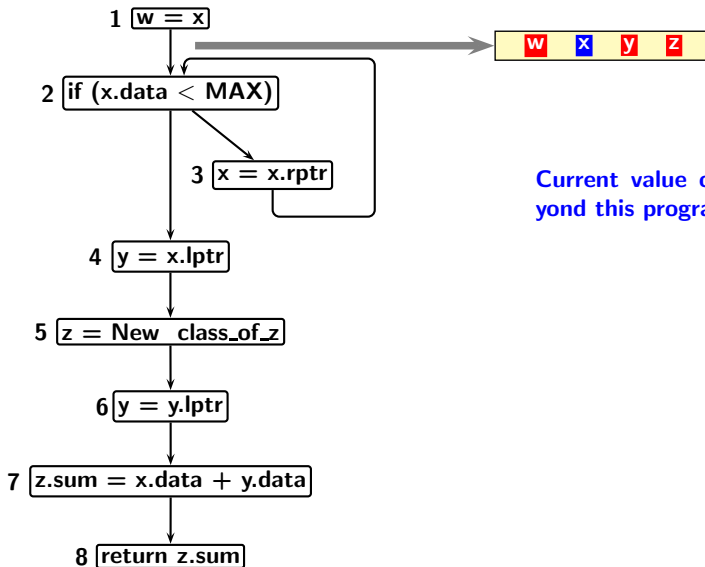
Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

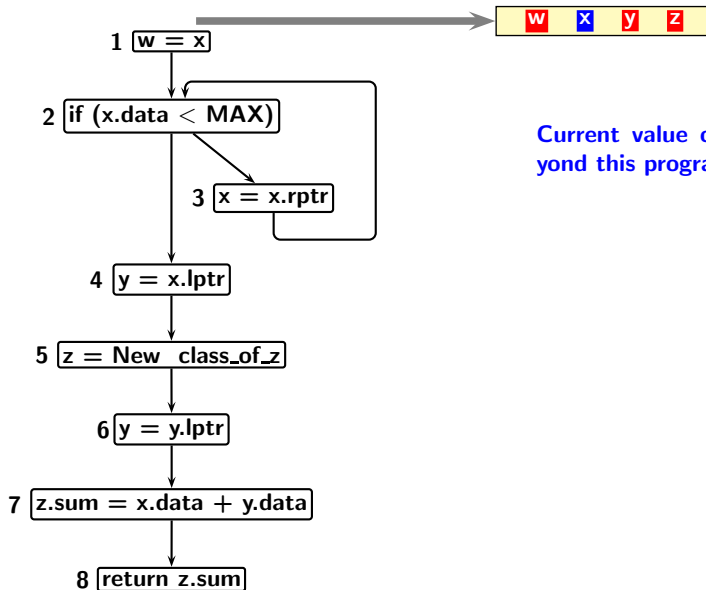
Live Variables
Analysis



Current value of x is used beyond this program point



Liveness of Stack Data: An Informal Introduction (2)



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



Liveness of Stack Data: An Informal Introduction (2)

IIT Bombay
cs302: Implementation
of Programming
Languages

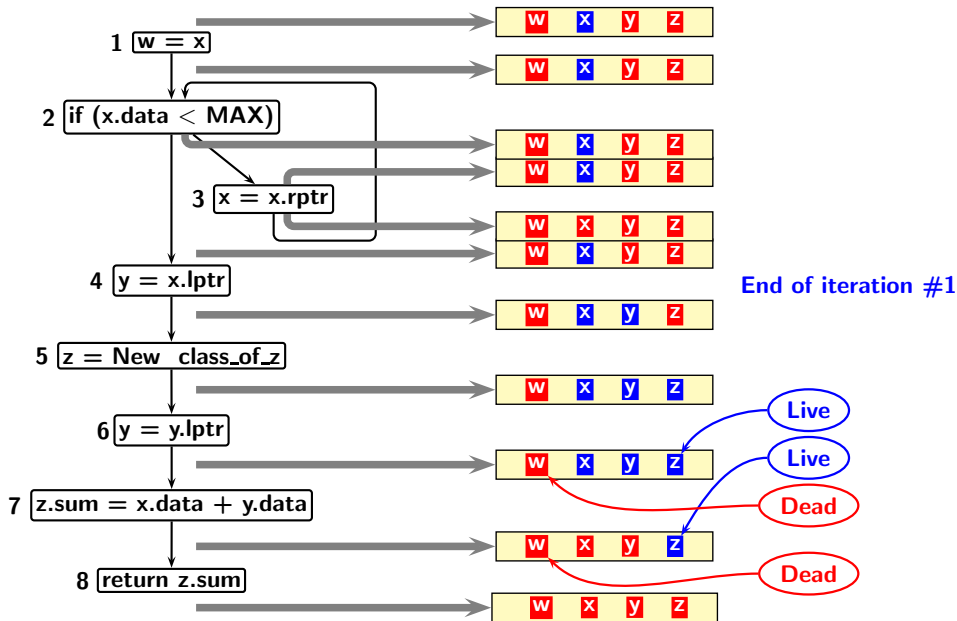
Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Liveness of Stack Data: An Informal Introduction (2)

IIT Bombay
cs302: Implementation
of Programming
Languages

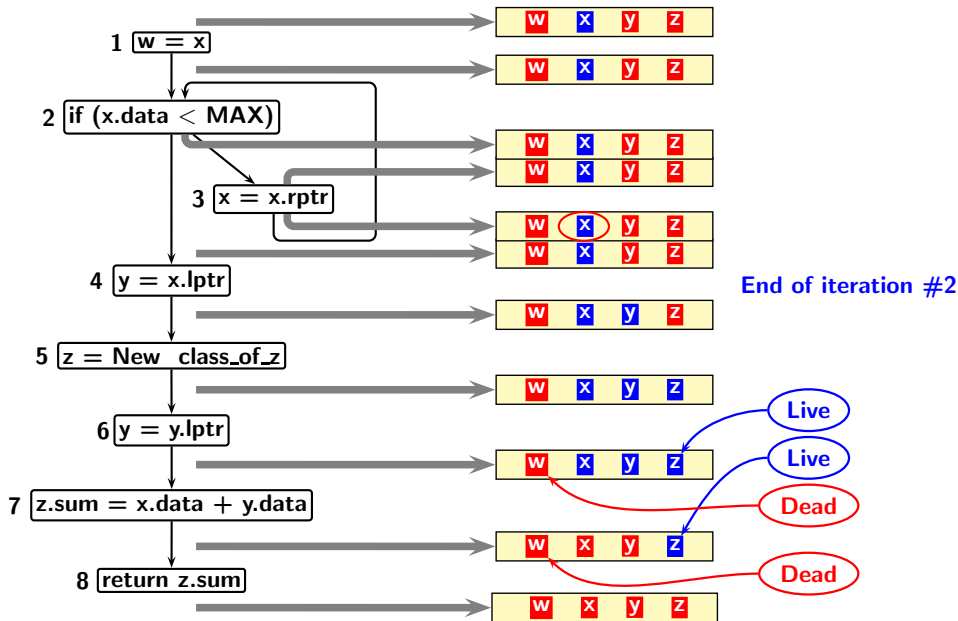
Topic:
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

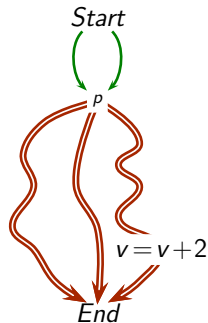
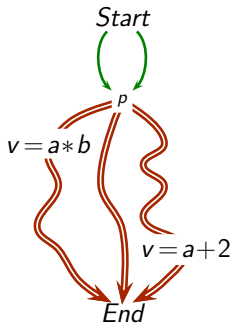
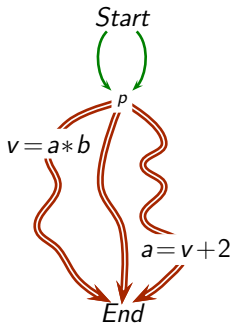
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Defining Live Variables Analysis

A variable v is live at a program point p , if **some** path **from p to program exit** contains an r-value occurrence of v which is not preceded by an l-value occurrence of v .

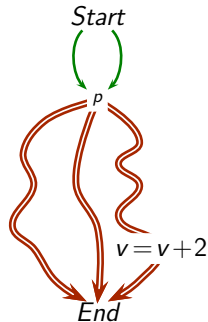
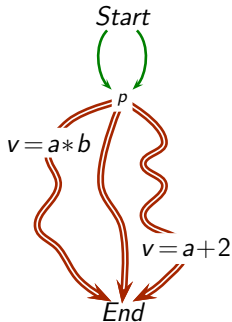
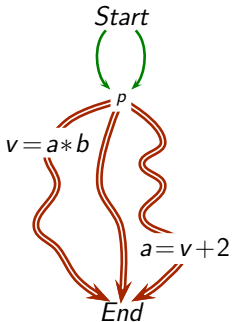




Defining Live Variables Analysis

A variable v is live at a program point p , if **some** path **from p to program exit** contains an r-value occurrence of v which is not preceded by an l-value occurrence of v .

v is live at p

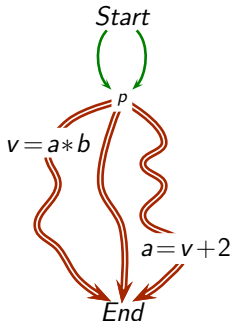




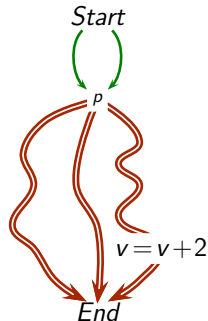
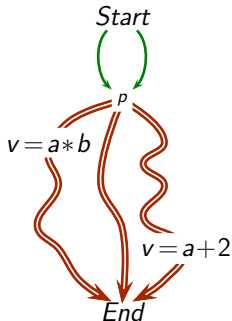
Defining Live Variables Analysis

A variable v is live at a program point p , if **some** path **from p to program exit** contains an r-value occurrence of v which is not preceded by an l-value occurrence of v .

v is live at p



v is not live at p



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



Defining Live Variables Analysis

A variable v is live at a program point p , if **some** path **from p to program exit** contains an r-value occurrence of v which is not preceded by an l-value occurrence of v .

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

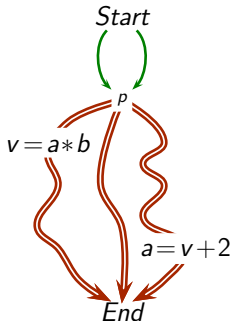
Section:

Classical
Optimizations

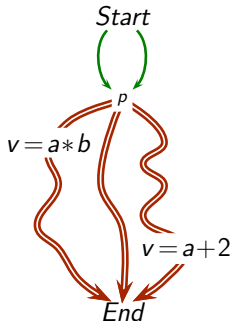
Optimizing Heap
Memory Usage

Live Variables
Analysis

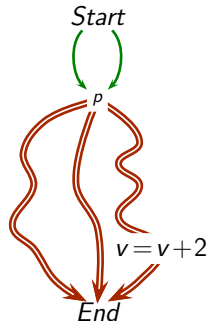
v is live at p



v is not live at p



v is live at p





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

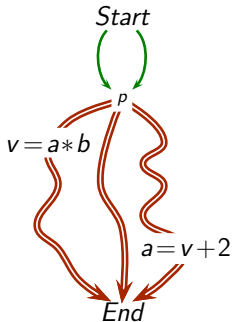
Live Variables
Analysis

Defining Live Variables Analysis

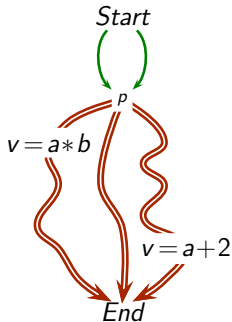
A variable v is live at a program point p , if **some** path **from p to program exit** contains an r-value occurrence of v which is not preceded by an l-value occurrence of v .

Path based
specification

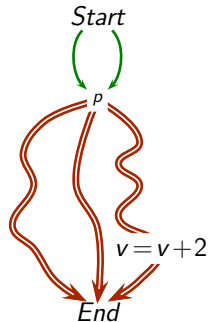
v is live at p



v is not live at p



v is live at p





Defining Data Flow Analysis for Live Variables Analysis

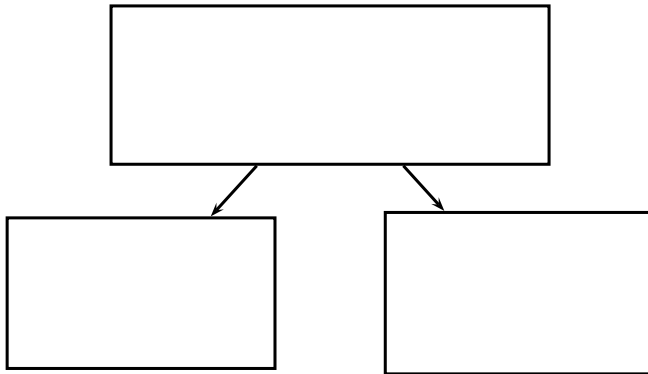
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

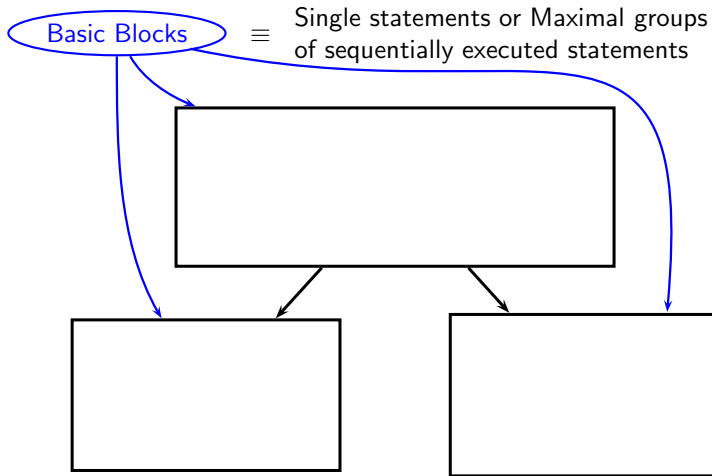
Optimizing Heap
Memory Usage

Live Variables
Analysis





Defining Data Flow Analysis for Live Variables Analysis



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

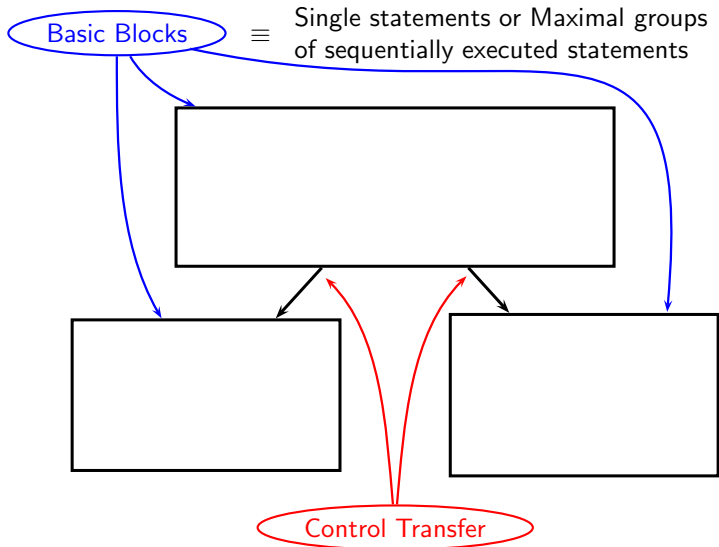
Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



Defining Data Flow Analysis for Live Variables Analysis



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis



Defining Data Flow Analysis for Live Variables Analysis

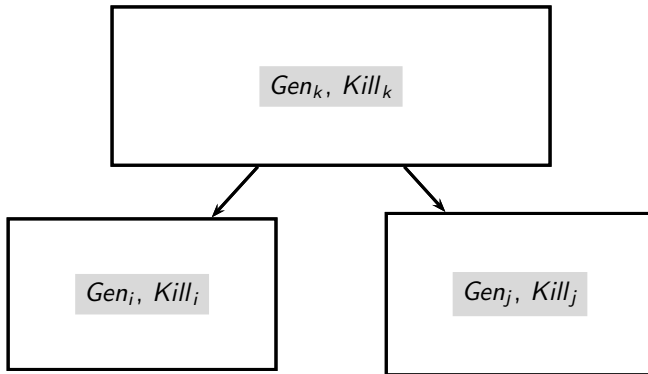
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Defining Data Flow Analysis for Live Variables Analysis

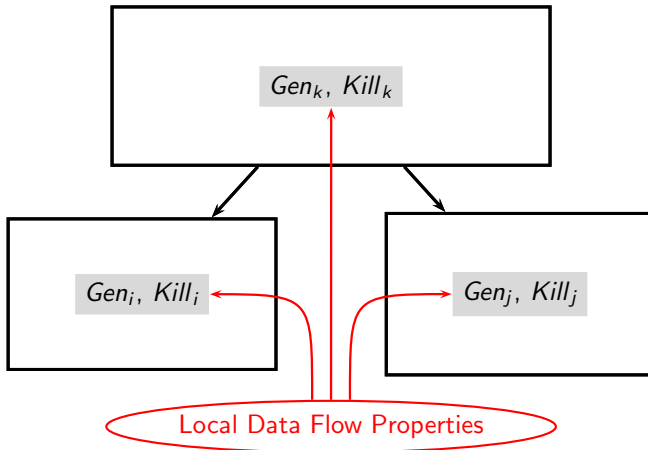
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Local Data Flow Properties for Live Variables Analysis

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

$$Gen_n = \{ v \mid \text{variable } v \text{ is used in basic block } n \text{ and} \\ \text{is not preceded by a definition of } v \}$$
$$Kill_n = \{ v \mid \text{basic block } n \text{ contains a definition of } v \}$$



Local Data Flow Properties for Live Variables Analysis

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

r-value occurrence

Value is only read, e.g. x, y, z in

$x.sum = y.data + z.data$

$Gen_n = \{ v \mid \text{variable } v \text{ is used in basic block } n \text{ and} \\ \text{is not preceded by a definition of } v \}$

$Kill_n = \{ v \mid \text{basic block } n \text{ contains a definition of } v \}$



Local Data Flow Properties for Live Variables Analysis

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Local Data Flow Properties for Live Variables Analysis

r-value occurrence

Value is only read, e.g. x,y,z in

`x.sum = y.data + z.data`

l-value occurrence

Value is modified e.g. y in

`y = x.lptr`

$Gen_n = \{ v \mid \text{variable } v \text{ is used in basic block } n \text{ and} \\ \text{is not preceded by a definition of } v \}$

$Kill_n = \{ v \mid \text{basic block } n \text{ contains a definition of } v \}$



Local Data Flow Properties for Live Variables Analysis

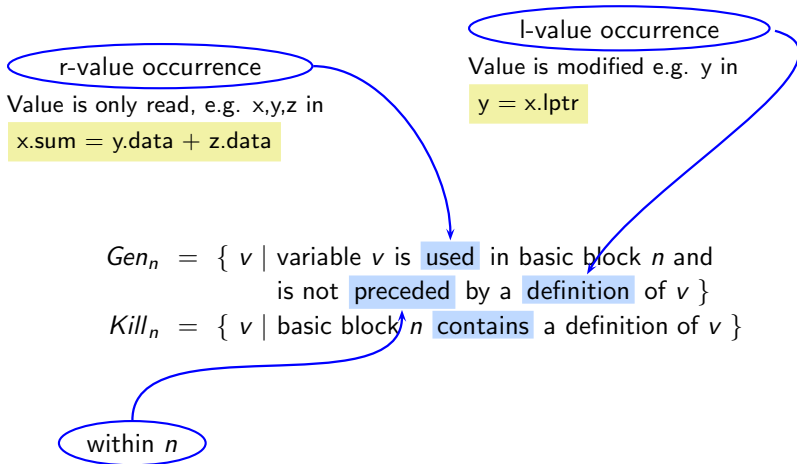
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Local Data Flow Properties for Live Variables Analysis

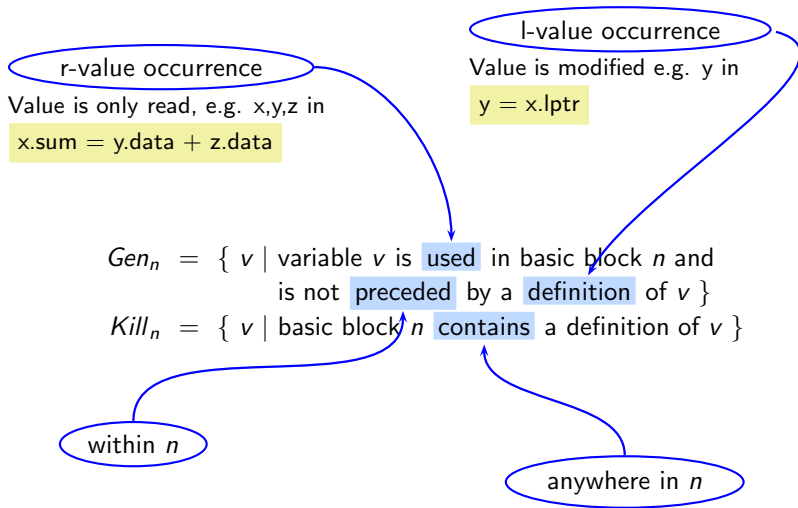
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Gen and Kill for Live Variables Analysis

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- What does Gen_n mean?
 - Set of variables with upwards exposed read within n
 - The most recent assignments to these variables are useful (and not dead)
 - The liveness of these variables is “generated” in node n
- What does $Kill_n$ mean?
 - Set of variables that are defined within n
 - The most recent assignments to these variables are not useful (are dead)
 - The liveness of these variables must be “killed” in node n
(i.e. liveness of these variables cannot propagate from the exit of node n to its entry)



Defining Data Flow Analysis for Live Variables Analysis

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

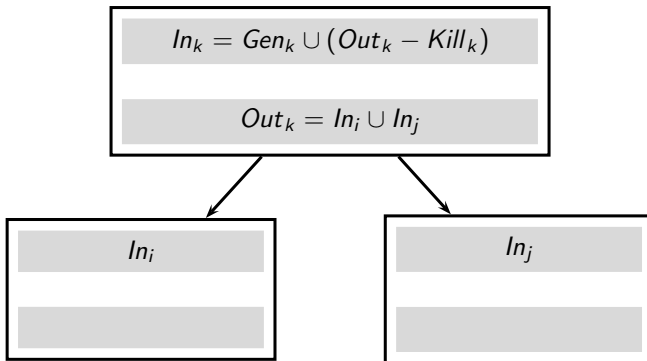
Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Defining Data Flow Analysis for Live Variables Analysis

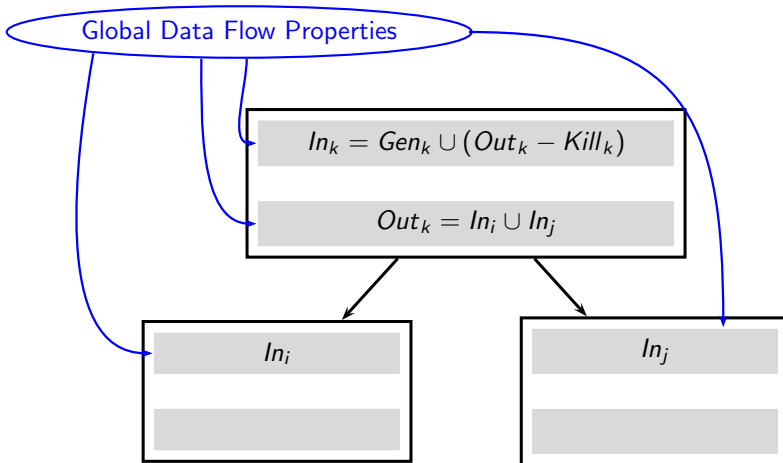
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Defining Data Flow Analysis for Live Variables Analysis

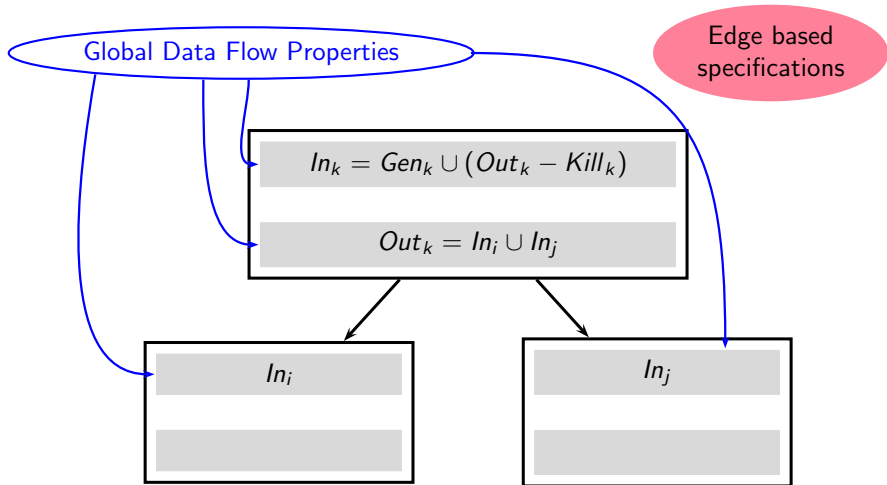
IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis





Data Flow Equations For Live Variables Analysis

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

$$In_n = (Out_n - Kill_n) \cup Gen_n$$

$$Out_n = \begin{cases} \text{ } & n \text{ is } End \text{ block} \\ \bigcup_{s \in succ(n)} In_s & \text{otherwise} \end{cases}$$

Alternatively,

$$\forall s \in succ(n). Out_n \supseteq In_s$$



Data Flow Equations For Live Variables Analysis

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

$$In_n = (Out_n - Kill_n) \cup Gen_n$$
$$Out_n = \begin{cases} BI & n \text{ is } End \text{ block} \\ \bigcup_{s \in succ(n)} In_s & \text{otherwise} \end{cases}$$

Alternatively,

$$\forall s \in succ(n). Out_n \supseteq In_s$$

- In_n and Out_n are sets of variables

Data Flow Equations For Live Variables Analysis



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

$$In_n = (Out_n - Kill_n) \cup Gen_n$$
$$Out_n = \begin{cases} BI & n \text{ is } End \text{ block} \\ \bigcup_{s \in succ(n)} In_s & \text{otherwise} \end{cases}$$

Alternatively,

$$\forall s \in succ(n). Out_n \supseteq In_s$$

- In_n and Out_n are sets of variables
- BI is boundary information representing the effect of calling contexts
 - $\emptyset$ for local variables (assuming 'return x' generates liveness of x)
 - set of global variables used further in any calling context (can be safely approximated by the set of all global variables)



IIT Bombay
cs302: Implementation
of Programming
Languages

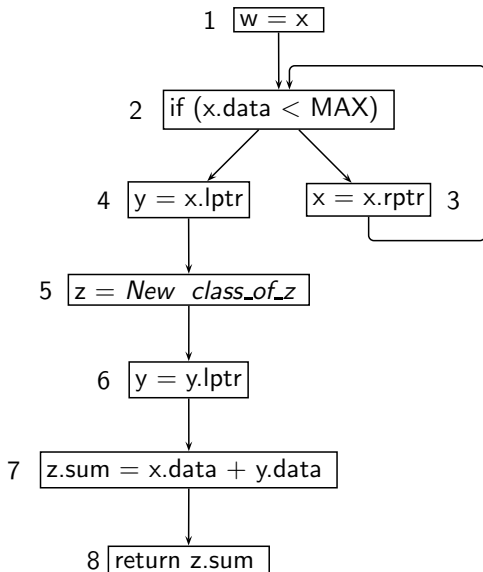
Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Data Flow Equations for Our Example



$$In_1 = (Out_1 - Kill_1) \cup Gen_1$$

$$Out_1 = In_2$$

$$In_2 = (Out_2 - Kill_2) \cup Gen_2$$

$$Out_2 = In_3 \cup In_4$$

$$In_3 = (Out_3 - Kill_3) \cup Gen_3$$

$$Out_3 = In_2$$

$$In_4 = (Out_4 - Kill_4) \cup Gen_4$$

$$Out_4 = In_5$$

$$In_5 = (Out_5 - Kill_5) \cup Gen_5$$

$$Out_5 = In_6$$

$$In_6 = (Out_6 - Kill_6) \cup Gen_6$$

$$Out_6 = In_7$$

$$In_7 = (Out_7 - Kill_7) \cup Gen_7$$

$$Out_7 = In_8$$

$$In_8 = (Out_8 - Kill_8) \cup Gen_8$$

$$Out_8 = \emptyset$$



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

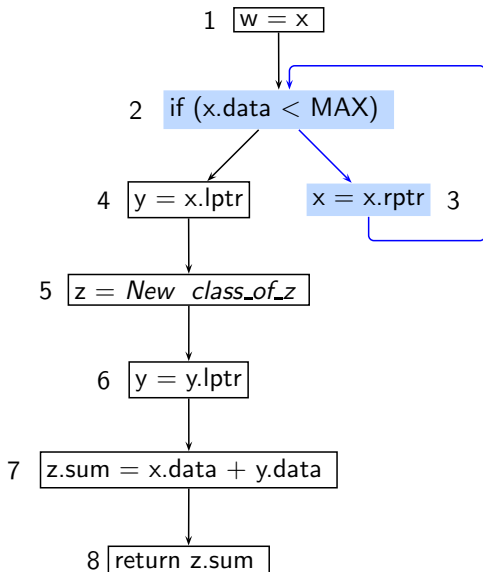
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Data Flow Equations for Our Example



$$In_1 = (Out_1 - Kill_1) \cup Gen_1$$

$$Out_1 = In_2$$

$$In_2 = (Out_2 - Kill_2) \cup Gen_2$$

$$Out_2 = In_3 \cup In_4$$

$$In_3 = (Out_3 - Kill_3) \cup Gen_3$$

$$Out_3 = In_2$$

$$In_4 = (Out_4 - Kill_4) \cup Gen_4$$

$$Out_4 = In_5$$

$$In_5 = (Out_5 - Kill_5) \cup Gen_5$$

$$Out_5 = In_6$$

$$In_6 = (Out_6 - Kill_6) \cup Gen_6$$

$$Out_6 = In_7$$

$$In_7 = (Out_7 - Kill_7) \cup Gen_7$$

$$Out_7 = In_8$$

$$In_8 = (Out_8 - Kill_8) \cup Gen_8$$

$$Out_8 = \emptyset$$



IIT Bombay
cs302: Implementation
of Programming
Languages

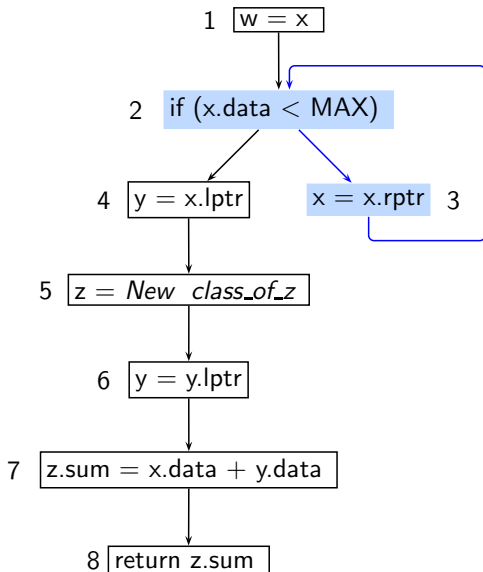
Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Data Flow Equations for Our Example



$$In_1 = (Out_1 - Kill_1) \cup Gen_1$$

$$Out_1 = In_2$$

$$In_2 = (Out_2 - Kill_2) \cup Gen_2$$

$$Out_2 = In_3 \cup In_4$$

$$In_3 = (Out_3 - Kill_3) \cup Gen_3$$

$$Out_3 = In_2$$

$$In_4 = (Out_4 - Kill_4) \cup Gen_4$$

$$Out_4 = In_5$$

$$In_5 = (Out_5 - Kill_5) \cup Gen_5$$

$$Out_5 = In_6$$

$$In_6 = (Out_6 - Kill_6) \cup Gen_6$$

$$Out_6 = In_7$$

$$In_7 = (Out_7 - Kill_7) \cup Gen_7$$

$$Out_7 = In_8$$

$$In_8 = (Out_8 - Kill_8) \cup Gen_8$$

$$Out_8 = \emptyset$$



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

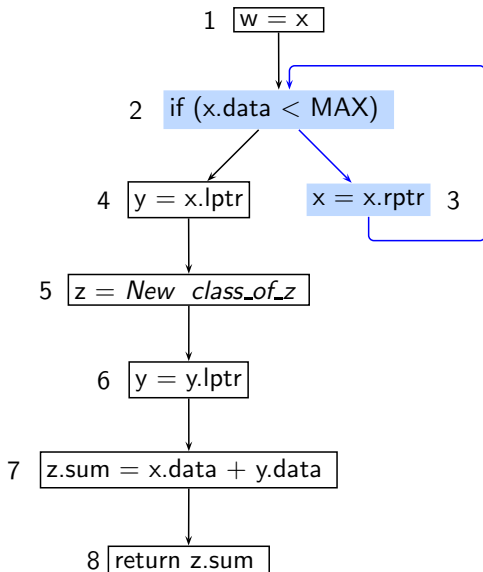
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Data Flow Equations for Our Example



$$In_1 = (Out_1 - Kill_1) \cup Gen_1$$

$$Out_1 = In_2$$

$$In_2 = (Out_2 - Kill_2) \cup Gen_2$$

$$Out_2 = In_3 \cup In_4$$

$$In_3 = (Out_3 - Kill_3) \cup Gen_3$$

$$Out_3 = In_2$$

$$In_4 = (Out_4 - Kill_4) \cup Gen_4$$

$$Out_4 = In_5$$

$$In_5 = (Out_5 - Kill_5) \cup Gen_5$$

$$Out_5 = In_6$$

$$In_6 = (Out_6 - Kill_6) \cup Gen_6$$

$$Out_6 = In_7$$

$$In_7 = (Out_7 - Kill_7) \cup Gen_7$$

$$Out_7 = In_8$$

$$In_8 = (Out_8 - Kill_8) \cup Gen_8$$

$$Out_8 = \emptyset$$



IIT Bombay
cs302: Implementation
of Programming
Languages

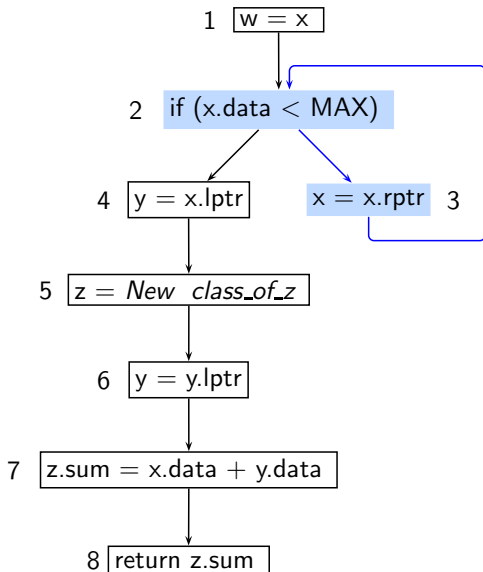
Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Data Flow Equations for Our Example



$$In_1 = (Out_1 - Kill_1) \cup Gen_1$$

$$Out_1 = In_2$$

$$In_2 = (Out_2 - Kill_2) \cup Gen_2$$

$$Out_2 = In_3 \cup In_4$$

$$In_3 = (Out_3 - Kill_3) \cup Gen_3$$

$$Out_3 = In_2$$

$$In_4 = (Out_4 - Kill_4) \cup Gen_4$$

$$Out_4 = In_5$$

$$In_5 = (Out_5 - Kill_5) \cup Gen_5$$

$$Out_5 = In_6$$

$$In_6 = (Out_6 - Kill_6) \cup Gen_6$$

$$Out_6 = In_7$$

$$In_7 = (Out_7 - Kill_7) \cup Gen_7$$

$$Out_7 = In_8$$

$$In_8 = (Out_8 - Kill_8) \cup Gen_8$$

$$Out_8 = \emptyset$$



IIT Bombay
cs302: Implementation
of Programming
Languages

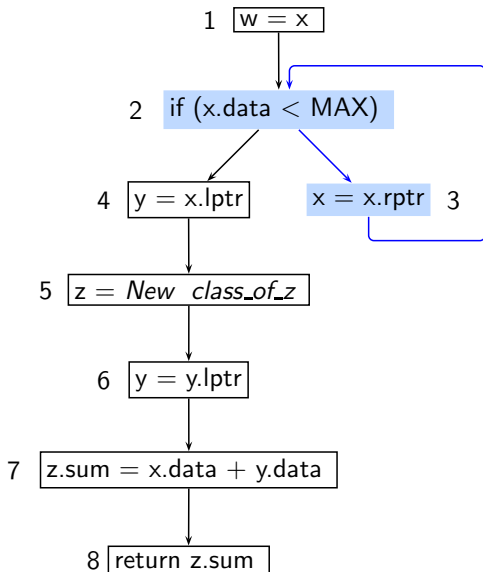
Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Data Flow Equations for Our Example



$$In_1 = (Out_1 - Kill_1) \cup Gen_1$$

$$Out_1 = In_2$$

$$In_2 = (Out_2 - Kill_2) \cup Gen_2$$

$$Out_2 = In_3 \cup In_4$$

$$In_3 = (Out_3 - Kill_3) \cup Gen_3$$

$$Out_3 = In_2$$

$$In_4 = (Out_4 - Kill_4) \cup Gen_4$$

$$Out_4 = In_5$$

$$In_5 = (Out_5 - Kill_5) \cup Gen_5$$

$$Out_5 = In_6$$

$$In_6 = (Out_6 - Kill_6) \cup Gen_6$$

$$Out_6 = In_7$$

$$In_7 = (Out_7 - Kill_7) \cup Gen_7$$

$$Out_7 = In_8$$

$$In_8 = (Out_8 - Kill_8) \cup Gen_8$$

$$Out_8 = \emptyset$$



IIT Bombay
cs302: Implementation
of Programming
Languages

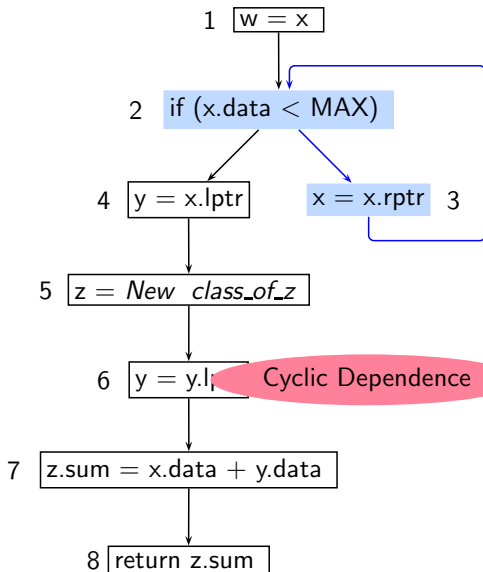
Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Data Flow Equations for Our Example



Cyclic Dependence

$$In_1 = (Out_1 - Kill_1) \cup Gen_1$$

$$Out_1 = In_2$$

$$In_2 = (Out_2 - Kill_2) \cup Gen_2$$

$$Out_2 = In_3 \cup In_4$$

$$In_3 = (Out_3 - Kill_3) \cup Gen_3$$

$$Out_3 = In_2$$

$$In_4 = (Out_4 - Kill_4) \cup Gen_4$$

$$Out_4 = In_5$$

$$In_5 = (Out_5 - Kill_5) \cup Gen_5$$

$$Out_5 = In_6$$

$$In_6 = (Out_6 - Kill_6) \cup Gen_6$$

$$Out_6 = In_7$$

$$In_7 = (Out_7 - Kill_7) \cup Gen_7$$

$$Out_7 = In_8$$

$$In_8 = (Out_8 - Kill_8) \cup Gen_8$$

$$Out_8 = \emptyset$$



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

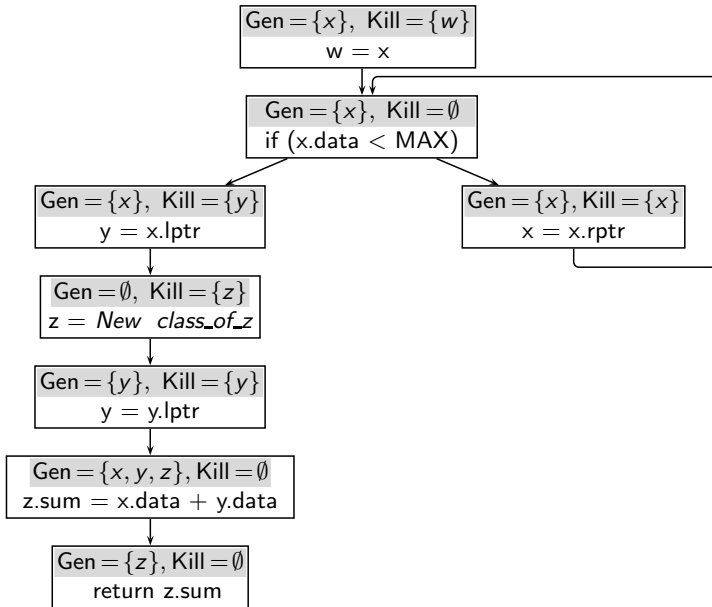
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Performing Live Variables Analysis





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

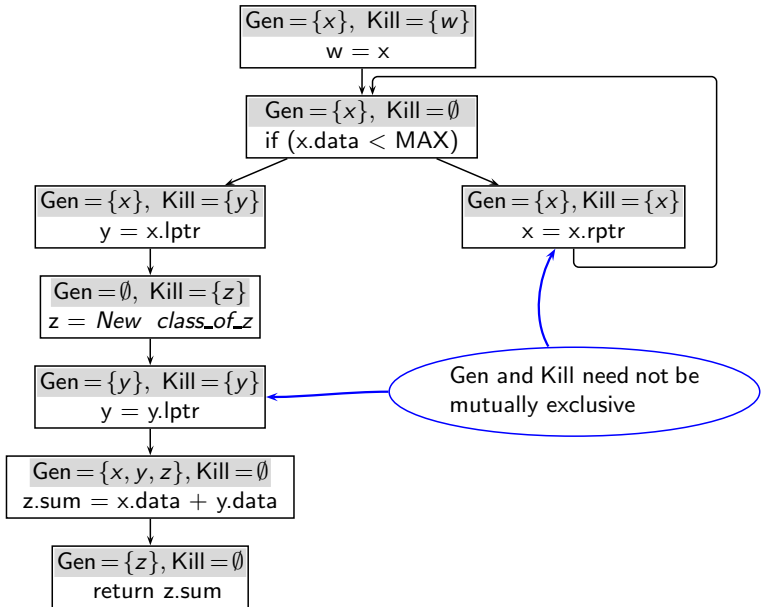
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Performing Live Variables Analysis





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

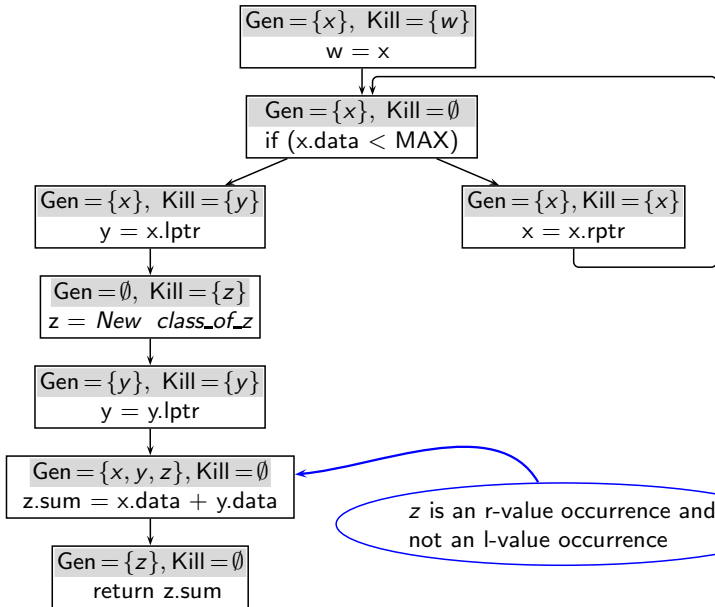
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Performing Live Variables Analysis





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

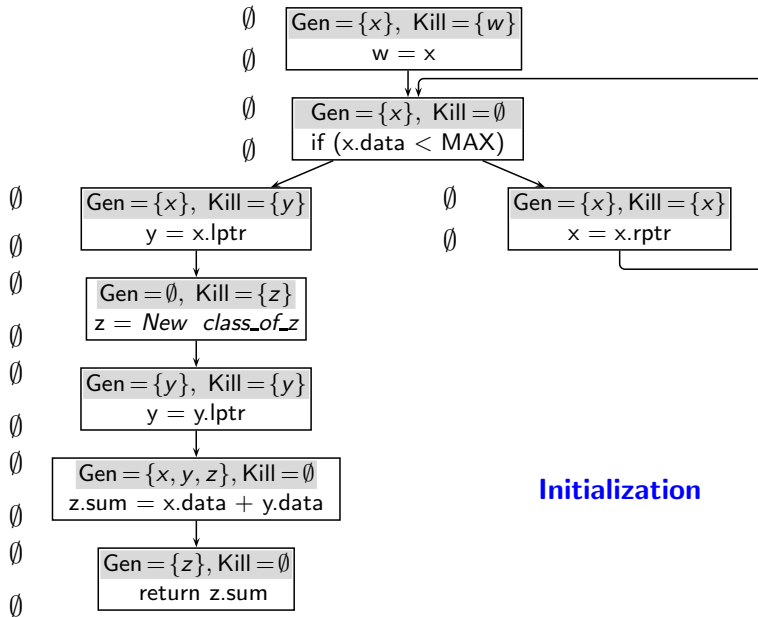
Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

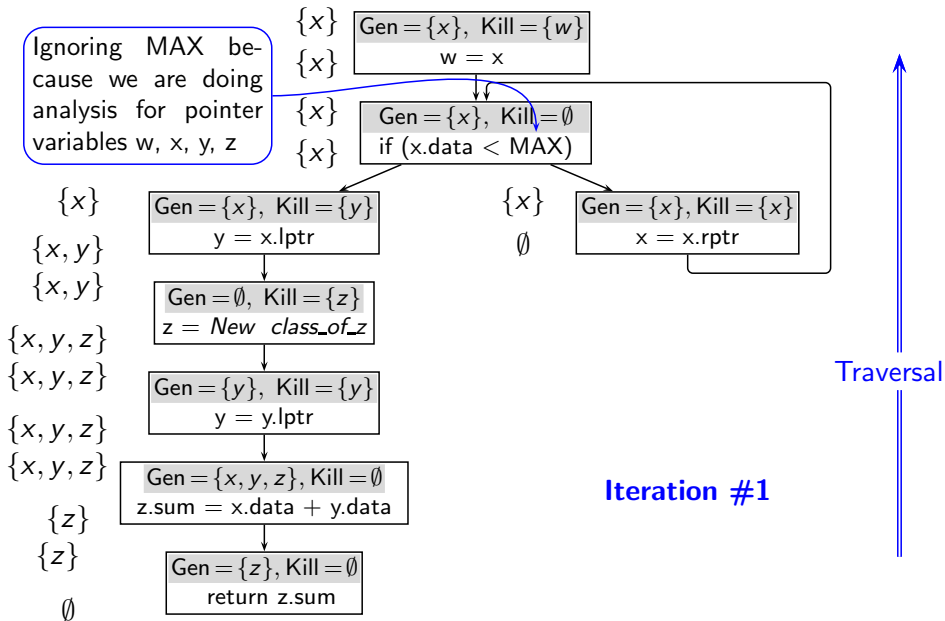
Performing Live Variables Analysis



Initialization

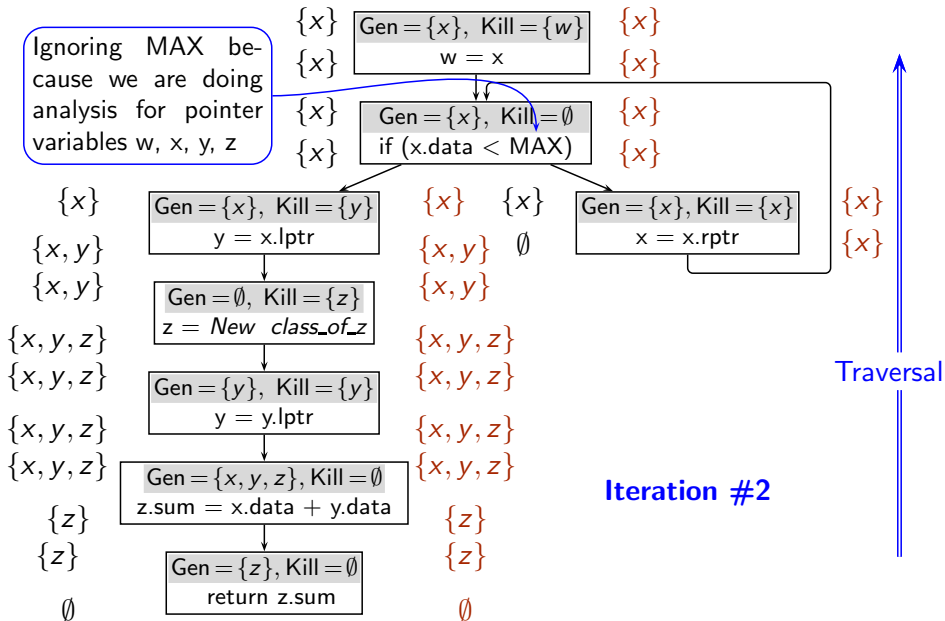


Performing Live Variables Analysis





Performing Live Variables Analysis





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:

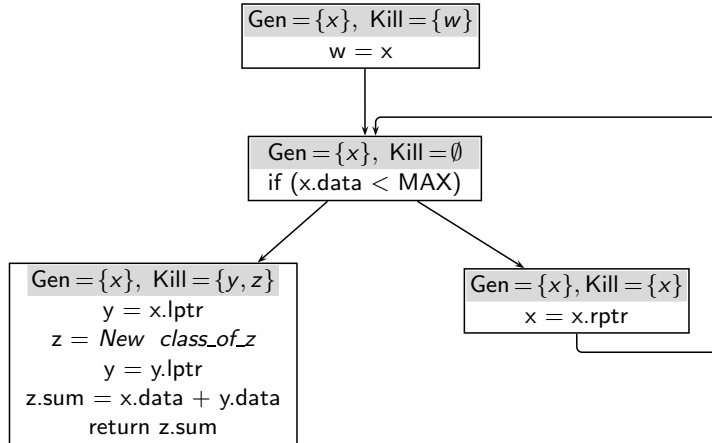
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

Performing Live Variables Analysis

Local data flow properties when basic blocks contain multiple statements



Using Data Flow Information of Live Variables Analysis



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Used for register allocation

If variable x is live in a basic block b , it is a potential candidate for register allocation

Using Data Flow Information of Live Variables Analysis



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Data Flow Analysis

Section:
Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Used for register allocation

If variable x is live in a basic block b , it is a potential candidate for register allocation

- Used for dead code elimination

If variable x is not live after an assignment $x = \dots$, then the assignment is redundant and can be deleted as dead code



What Does Data Flow Analysis Involve?

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Defining the analysis.
- Formulating the analysis.
- Performing the analysis.



What Does Data Flow Analysis Involve?

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Defining the analysis. Define the properties of execution paths
- Formulating the analysis.
- Performing the analysis.



What Does Data Flow Analysis Involve?

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- **Defining the analysis.** Define the properties of execution paths
- **Formulating the analysis.** Define data flow equations
 - Linear simultaneous equations on sets rather than numbers
 - Later we will generalize the domain of values
- **Performing the analysis.**



What Does Data Flow Analysis Involve?

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- **Defining the analysis.** Define the properties of execution paths
- **Formulating the analysis.** Define data flow equations
 - Linear simultaneous equations on sets rather than numbers
 - Later we will generalize the domain of values
- **Performing the analysis.** Solve data flow equations for the given program flow graph



What Does Data Flow Analysis Involve?

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- **Defining the analysis.** Define the properties of execution paths
- **Formulating the analysis.** Define data flow equations
 - Linear simultaneous equations on sets rather than numbers
 - Later we will generalize the domain of values
- **Performing the analysis.** Solve data flow equations for the given program flow graph
- Many unanswered questions
Initial value? Termination? Complexity? Properties of Solutions?

Our Method of Performing Data Flow Analysis



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Data Flow Analysis

Section:

Classical
Optimizations

Optimizing Heap
Memory Usage

Live Variables
Analysis

- Round robin iteration using the Jacobi method
(use the values from the current iteration wherever possible)
- Unknowns are the data flow variables In_i and Out_i
- Domain of values is not numbers
- Computation in a fixed order
 - either forward (reverse post order) traversal, or
 - backward (post order) traversal

over the control flow graph

We assume that

- every node is reachable from $Start$, and
- End is reachable from every node